

User Manual for MPEC version 2.1 theta

David W. Schwenke*
NASA Ames Research Center
Mail Stop 258-2, P.O. Box 1
Moffett Field, CA 94035-0001

April 25, 2025

The logo for the Minor Planet Center (MPEC) is displayed in a bold, black, sans-serif font. The letters 'm', 'p', and 'e' are lowercase, while 'c' is uppercase. The 'm' and 'p' are connected, as are the 'e' and 'c'.

*E-mail: david.w.schwenke@nasa.gov, david.schwenke@gmail.com

Contents

1	Introduction	4
2	Input	5
3	BG	6
3.1	<i>rel</i>	6
3.2	geometry	6
3.2.1	vibrational coordinates	7
3.2.2	Cartesian coordinates	8
3.3	basis set	8
3.3.1	basis library	8
3.3.2	read in	9
3.4	relativistic contraction	10
3.5	symmetry	10
3.6	finite field	10
4	SOHF	10
5	CAS	11
5.1	<i>guess</i>	11
5.2	occupations	11
5.3	<i>states</i>	12
5.3.1	term symbol	12
5.4	dynamic weights	13
5.5	<i>molden</i>	13
5.6	<i>rydberg</i>	13
5.7	<i>grad</i>	15
5.8	miscellaneous keywords	15
6	DIAB	16
7	CAT	16
8	MRCI	17
8.1	occupancies	18
8.1.1	occupancy restrictions	18
8.2	<i>state</i>	18
8.3	excitations	18
8.4	internal contraction	19
8.5	restarts	19
8.6	miscellaneous keywords	19
9	PROP	20
10	<i>debug</i>	21

11 Sample inputs	22
11.1 C atom basis contraction	22
11.2 Ne atom	23
11.3 He atom	24
11.4 N atom	26
11.5 HF molecule	28
11.6 OH molecule	31
11.7 NO ⁺ molecule	32
11.8 C ₂ molecule	35
11.9 CH ₂ molecule	36
11.10O ₂	44
11.11CH ₄	45
11.12N ₂	46
11.13CN	48
11.14HCN	48
12 Release Notes	49
12.1 version theta	49
12.2 version eta	50
12.3 version zeta	50
13 Installation	50

1 Introduction

Welcome to Massively Parallel Electron Correlation, version 2.1 theta! When using MPEC, please cite D. W. Schwenke, *J. Chem. Phys.* **158**, 084801 (2023).

MPEC consists of a series of programs: *i* routines for evaluating one and two electron integrals over Gaussian type basis functions, *ii* routines for optimizing molecular orbitals, *iii* routines for computing correlation energies, and *iv* routines for computing property matrix elements of the wavefunctions. Molecular point groups consisting of one to three reflections or an inversion can be utilized. However, only pairs of equivalent nuclei are allowed. MPEC is parallelized using MPI routines, and does not store intermediate quantities on disk.

Most of the MPEC code was written in FORTRAN specifically for MPEC, however we use some utility programs from other sources. These include the file `njsymb16.f`, which includes the $3-j$ symbol routine of Caswell and Maximon¹, the sorting routine `DSORT` by Jones and Wisniewsk², and the Gaussian quadrature routine `GAUSSQ`³. These routines are all in the distribution package. We also utilize numerous routines from LAPACK: hopefully your computer vendor supplies optimized versions of these routines, however in case they do not, we have provided copies of these routines from `netlib.org`. The installer is assumed provide their own FORTRAN compiler and a MPI library.

The integral code can handle Gaussian basis functions with a maximum angular momentum of 15 for non-relativistic calculations. General contraction of the basis functions is available. Relativistic effects can be included using the Spin-Free method of Dyll.⁴

The routine for optimizing molecular orbitals is a powerful energy minimization algorithm, and can do closed shell Hartree-Fock wave functions as well as Multi-configuration Hartree-Fock (MCHF) wave functions with zero or more doubly occupied orbitals and zero or more active orbitals in which a full Configuration Interaction calculation is performed. It is also possible to reduce the number of the configuration state functions formed from the active orbitals by restricting the maximum number of open shells. In the case of MCHF wave functions, the energy minimized can be the state average of multiple energies, with fixed or dynamic weights. Rydberg orbitals can also be determined once the valence orbitals are determined.

The correlation energies are computed using Multi-Reference Configuration Interaction with the N-2 electron functions either uncontracted or contracted.

Besides standard input and standard output, several other named files can be used. Initial guess orbitals to the orbital optimization routine are supplied via the file `orbs` and the final optimized orbitals are saved in the file `forbs`. If the optimization does not converge, the best results so far are saved in the file `borbs`. These three files all have the same structure, so `forbs` or `borbs` can be copied to `orbs` and used as an initial guess in a subsequent calculation. There is

¹NBS Technical note 409

²`netlib.org`

³`netlib.org`, attributed to Stanford University with modifications by Eric Grosse

⁴*J. Chem. Phys.*, 100, 2118-2127 (1993)

also an option to dump the orbitals and basis set information to the file *molden* using molden format. If MCHF calculations are carried out, one has the option of writing to the file *summarycas*: this contains the geometric data as well as the final energies. If an MRCI calculation is carried out, the orbitals will be taken from the file *forbs* and the wavefunction will be saved in the file *wavef* and a summary of the calculation saved in the file *summary*. The creation of the file *wavef* can be turned off if no properties are required. Restart information can be saved to *wavef*. The *wavef* file also includes the orbitals, so it contains all the data required for property calculations.

Important information is indicated in the output by the > character. This information from the MRCI routine is also saved in the *summary* file. In case of disaster, individual processors will also write to the files *output.1*, *output.2*, ...

Memory usage per processor is limited by the parameter *maxbc*, and by default is set to $2^{31} - 1$, the maximum amount of memory addressable using signed 4-byte integers. This limit can exceed the physical memory available without detriment unless the calculation actually tries to use more memory than is available.

2 Input

All inputs having units are given in Hartree atomic units, unless specified otherwise.

The input for the different program sections are separated by section names, preceded by an ampersand (&). The allowed section names are: BG (basis and geometry), SOHF, CAS, DIAB, CAT, MRCI, and PROP. These must be given in uppercase. Within each program section, several different inputs are possible through the use of keywords. These keywords must be in lowercase. If a keyword is not given, sensible defaults will be used. In all cases except one, the input for a given keyword is on the same line as the keyword. If multiple values are given, they should be delimited with at least one space or a comma. The one exception is the *states* keyword in the CAS program section. The reason for this one exception is there are an unknown number of states to be input.

The order of the program sections should be BG, then SOHF or CAS, then MRCI, and perhaps PROP. If orbitals are already available, one does not need BG, for that data is stored in the orbital file. If MRCI calculations have been carried out, only the PROP program section is required, for the needed information is stored in the *wavef* file. If diabatic orbitals are formed using DIAB, then only that section name will be considered.

To skip a calculation but retain the keywords in the input file, place the character # in column one of the data lines (including the section line) and that part of the input will be skipped over. This same character can also be used to delimit comments at the end of a line in the MPEC input. See the sample inputs for examples of this.

3 BG

In this section of input, one specifies the nuclear types, positions, and Gaussian basis sets. Also, whether or not relativistic effects are included, and if there are any external electric fields.

Table 1: BG quick reference

<i>alpha</i>	<i>ctheta</i>	<i>ends</i>	<i>ffield</i>	<i>grab</i>
<i>len</i>	<i>phi</i>	<i>r</i>	<i>rel</i>	<i>sym=off</i>
<i>theta</i>				

3.1 *rel*

To turn on relativistic effects, use the keyword *rel*. This keyword must appear before the nuclear position input.

The arguments to this keyword are either *off*, meaning relativistic effects are not included, *dhf* meaning a Dirac-Hartree-Fock calculation is carried out, and/or the type of two electron integrals to use. The type can be *ss* or *ssss*. If the type is *ss*, the integrals proportional to $1/c^4$ will be neglected, while if the type is *ssss*, they will be included. The type can also be appended by *g* or *b*, in which case after the spinars are converged, the Gaunt or Breit interaction will be computed. In the case of a Dirac-Hartree-Fock calculation, this is done immediately after the energy calculation, while in the Spin-Free case, this is done by the property program. Whether or not to include the Gaunt or Breit interaction can also be controlled by a keyword in the property program input. The Gaunt or Breit interaction is never used during the optimization of the wavefunction.

One can also redefine the fine-structure constant $\alpha = 1/137.0..$ using the *alpha* keyword. This is useful if one is trying to make the relativistic calculations closer to the non-relativistic limit, although this does not change the finite size of the nuclei. Thus the limit $\alpha \rightarrow 0$ will not reproduce the non-relativistic calculations, for the former uses finite-sized nuclei and the later uses point nuclei when evaluating the nuclear attraction integrals.

3.2 *geometry*

The nuclear positions can be specified in one of two ways: the Cartesians can be explicitly read in or vibrational coordinates can be used. To use other than atomic units for distances, use the *length* keyword. Then the input coordinates will be multiplied by the argument of this keyword before being used. One can input a number or a ratio, *e.g.*

```
length 1/0.529177249
```

will convert the coordinates from Å to atomic units.

3.2.1 vibrational coordinates

In this case, the vibrational coordinates must be defined before numerical values are given. This is done by including one or more lines defining internal vectors: e.g.

ends(1)=C to AB, ends(2)=A to B

sets up Jacobi vectors with the first vector going from nuclei C to the center of mass of AB. For triatomics, one can also use bond-length-bond-angle coordinates via

ends(1)=A to B, ends(2)=B to C

or Radau coordinates with

ends(1)=A to ABC, ends(2)=B to ABC

with C being the heavy nuclei. For more than three nuclei, just add more vectors, e.g. for CO₂+CO₂ using a mixture of Radau and Jacobi coordinates, use

ends(1)=ABC to DEF, ends(2)=A to ABC, ends(3)=B to ABC,
ends(4)=D to DEF, ends(5)=E to DEF

If other than the most common isotopologue is desired, put the desired isotopologue mass number in front of the Atomic symbol, *i.e.* 18O.

Unless modified by the *bisect* keyword, the nuclear center of mass of the system will be placed at the origin, the z axis will be along the first vector, and the second vector will lie in the xz plane.

Then the spherical polar values for the vectors need to be given: via

r=value1, value2, ... valuen

theta=angle1, angle2, ... anglen-1

phi=az1, az2, ... az n-2

where n is the number of vectors, or one less than the number of nuclei. With the above input, theta and phi are input in radians. If thetad and/or phid are used, the angles are input in degrees. If ctheta is used, the cosine of theta is to be input.

For a triatomic, one can use *bisect* (or *bisectx*) for the first and second vector to bisect the x axis, *i.e.* if thetad=180°, the molecule will line on the z axis, or one can use *bisectz* for the first and second vector to bisect the z axis.

These spherical polar values can be in the input file, or one can replace them with the line

grab *lfn*

where *lfn* is a disk file name.

After the spherical polar parameters are read in, include a series of lines specifying the nuclei and what symbol it corresponds to the vector definition,

e.g. for the CO₂+CO₂ example

```
O is A,B,D,E
basis input
C is C,F
basis input
```

The basis input is discussed in 3.3 below.

It should be noted that the Atomic symbol must be capitalized as usual, *i.e.* He, not he or HE for helium.

If the calculation is for an atom, no geometric input is needed, except if the Gaussian basis is input explicitly, in which case one needs to specify the Cartesian coordinates.

3.2.2 Cartesian coordinates

If explicit Cartesians are to be input, use for each nuclei

line 1: Atomic symbol or Z= atomic number
line 2 to 1+n: x,y,z coordinates in atomic units for atoms 1 to n of the type specified by line1,
line 2+n to 2+m: Gaussian basis set information.

These lines are repeated for each nuclei type. The coordinate input **MUST** contain decimal points.

Alternatively, if vibrational coordinates specify the geometry, use for each nuclei

line 1: Atomic symbol (or Z=#) is A (and/or B,C, ...)
line 2 to 2+m: Gaussian basis set information.

It should be noted that the Atomic symbol must be capitalized as usual, *i.e.* He, not he or HE for helium.

3.3 basis set

The Gaussian basis set information can be given explicitly or by means of a file name.

3.3.1 basis library

The valid file names are found in the *data* directory, and currently includes the correlation consistent basis sets of Dunning and coworkers: these are called cc-pvdz, cc-pvtz, cc-pvqz, cc-pv5z, and cc-pv6z. For the second row, there are also the basis sets with an extra d function: cc-pv(d+d)z, cc-pv(t+d)z, etc. There are also files like cc-pvtz.aug in the *data* directory. These files just contain the extra functions for adding diffuse functions. These file are easily generated: for cc-pvtz.aug, I used the unix command

```
diff aug-cc-pvtz cc-pvtz > cc-pvtz.aug
```

In general, one can have *e.g.* the file cc-pvtz.xyz that could have what ever functions are of interest, *i.e.* diffuse functions, core-valence functions, *etc.* To use these extra functions, add &xyz after the file name in the input. More than

one addition is possible, *i.e.* `cc-pvtz &aug &xyz`. Core-valence functions must have `cv` as part of `xyz`: this is because relativistic calculations need to know that assuming a non-relativistic contraction of the individual tight Gaussians is not the way to eliminate positron states.

In addition, on the same line as the file name, extra directions can be given. These start with `+`, `-`, or `u`. The symbol `+` causes extra diffuse basis functions to be added for each angular momentum in the basis. One can also add selected diffuse functions, *e.g.* to only add diffuse `sp` functions, use `+sp`. To add two diffuse functions, use *e.g.* `+2sp` which means add two diffuse `s` functions and two diffuse `p` functions. An even tempered set of exponential parameters will be used. If there are at least two Gaussian exponential parameters in the basis for the symmetry that is being augmented, the even tempered ratio used will be computed using the two most diffuse functions already in the basis. If only one exponential parameter is present, the ratio 2.5 will be used.

The symbol `-` causes basis function symmetries to be deleted: *e.g.* to use the `sp` part of the `tz` basis set, use `cc-pvtz -df`. Finally, `u` causes the basis set to be uncontracted. To uncontract all the functions, just use `u`, while to say, just uncontract the `s` functions, use `us`.

It should be noted that the `d-aug` sets of Dunning and coworkers can be used by combining the `.aug` file with `+`, *i.e.*

```
cc-pvtz &aug +2sp
```

Additional basis set files can simply be added by downloading them from the MolSSI ESSL website - select the NWChem basis format.

Citation

When publishing results obtained from use of the Basis Set Exchange software, please cite:

A New Basis Set Exchange: An Open, Up-to-date Resource for the Molecular Sciences Community. Benjamin P. Pritchard, Doaa Altarawy, Brett Didier, Tara D. Gibson, Theresa L. Windus. *J. Chem. Inf. Model.* 2019, 59(11), 4814-4820, doi:10.1021/acs.jcim.9b00725.

For citing the previous EMSL/PNNL Basis Set Exchange, please cite the following references:

The Role of Databases in Support of Computational Chemistry Calculations Feller, D., *J. Comp. Chem.* 1996, 17(13), 1571-1586. Basis Set Exchange: A Community Database for Computational Sciences Schuchardt, K.L., Didier, B.T., Elsethagen, T., Sun, L., Gurumoorthi, V., Chase, J., Li, J., and Windus, T.L. *J. Chem. Inf. Model.* 2007, 47(3), 1045-1052, doi:10.1021/ci600510j.

3.3.2 read in

If the Gaussian basis set information is read in explicitly, it is given as follows: for each angular momentum, input a line with ℓ , *nzeta*, *ncont*. *nzeta* is the number of exponential parameters, and *ncont* is the number of contracted functions. If *ncont* is omitted, the basis set is not contracted. Next input a line with the *nzeta* exponential parameters. Then, if contraction is used, input *ncont* lines,

each with the *nzeta* contraction coefficients, or if a relativistic calculation is being done, each line will have 2 *nzeta* contraction coefficients, the first *nzeta* for the k=1 and the second for k=2.

The nuclei input is terminated with a program section that starts with & in column one of the input, *i.e.* one of the subsequent program sections.

3.4 relativistic contraction

For relativistic calculations with contracted basis sets, one needs to generate the contraction coefficients if I have not done so already. The procedure I use is to run calculations with uncontracted basis sets to reproduce the non-relativistic contraction coefficients, then run the same calculation with relativity turned on. Then in the basis input file, after the non-relativistic contraction coefficients, add a space, "rc", another space, then the relativistic contraction coefficients: the ao coefficients printed out by the CAS code. See the C atom sample calculation.

3.5 symmetry

The molecular point group symmetry will be automatically determined by the program. To force the use of no symmetry, include the line *sym=off* in this part of the input.

3.6 finite field

To include an electric field in the the calculation, use the keyword *ffield*. The arguments for this keyword are *type* and *strength*. The field strength in a.u. is given by *strength*, and the type by *type*, which is *x*, *y*, *z*, for electric dipole moments, and *xx*, *xy* *etc* for the quadrupole moment. One can include more than one field type on the *ffield* line, *e.g.* to do a dipole polarizability or the spherical part of the quadrupole moment.

4 SOHF

In this section, parameters for controlling the Hartree-Fock calculation are given. The final converged orbitals will be saved in the file *forbs*. If the calculation did not converge, the best orbitals will be saved in the file *borbs*.

Table 2: SOHF quick reference

<i>debug</i>	<i>nocan</i>	<i>grad</i>	<i>gues</i>	<i>wern</i>
<i>davi</i>	<i>char</i>	<i>nocc</i>	<i>occ</i>	<i>maxit</i>

The useful keywords for this section are *charge*, *guess* and *grad*. None are required.

The keyword *charge* specifies the overall charge of the system. If *charge* is not specified, it is assumed to be zero or +one, depending on which choice gives

an even number of electrons. The number of occupied molecular orbitals is half the number of electrons. The occupied orbitals will be assigned to symmetry blocks based on the eigenvalues of the one-electron hamiltonian matrix.

The keyword *guess* specifies the starting orbitals. The allowed values are *h0* and *readin*. The default is *h0*, *i.e.* use the eigenvectors of the one-electron hamiltonian matrix. If *readin* is specified, the program will read in previous orbitals from the file *orbs* and start from them. The guess routine can take orbitals from different geometries, different symmetries, or from a different one-electron basis set.

5 CAS

In this section, the parameters for the calculation of the MCHF orbitals are given. The final orbitals will be saved on the file *forbs*. If the calculation did not converge, the best orbitals will be saved on the file *borbs*.

Table 3: CAS quick reference

<i>3rd</i>	<i>act</i>	<i>csf</i>	<i>debug</i>	<i>doub</i>
<i>dynw</i>	<i>econv</i>	<i>epssym</i>	<i>first</i>	<i>gconv</i>
<i>grad</i>	<i>guess</i>	<i>maxitci</i>	<i>maxitmac</i>	<i>maxps</i>
<i>molden</i>	<i>nocan</i>	<i>nowern</i>	<i>npdiag</i>	<i>occ</i>
<i>pthrs</i>	<i>ryd</i>	<i>sneglect</i>	<i>states</i>	<i>sum</i>
<i>usenormal</i>	<i>vconv</i>	<i>wern</i>		

The only required keyword is *states*.

5.1 *guess*

The keyword *guess* specifies the initial guess for the orbitals. The allowable values are *h0* or *readin*. If *guess* is not included, the program will assume *guess readin*. If *guess* is given without an argument, *guess h0* will be used. To start from HF orbitals, one has to have run a separate calculation using the SOHF routine, *i.e.* one can not piggy back the CAS calculation on a SOHF calculation. The initial guess does not need to be from a calculation with the same geometry, the same symmetry, or the same basis set.

5.2 occupations

The keyword *double* specifies the number of doubly occupied orbitals in each symmetry. If this keyword is included, but the rest of the line is left blank, the default set of orbitals will be used.

The keyword *active* specifies the number of active orbitals in each symmetry. If this keyword is included, but the rest of the line is left blank, the default set of orbitals will be used.

The keyword *occup* specifies the sum of the number of doubly occupied and active orbitals in each symmetry. If this keyword is included, but the rest of the line is left blank, the default set of orbitals will be used. The use of *double* and *active* is pretty clear, but the inclusion of *occup* might seem superfluous. An example where it is useful is in cases with *e.g.* oxygen atoms, where the 2s-like orbital will be essentially always be doubly occupied. Then one can use *double* in conjunction with the default *occup* to specify the active space.

If none of *double*, *active*, or *occup* is given, the defaults will be used.

5.3 *states*

The keyword *states* triggers the reading in a series of subsequent lines specifying the electronic states.

Each line consists of *ichrg,termsym,nroot,(wgt(i),i=1,nroot)*

where *ichrg* is the overall charge, *termsym* is the term symbol, specifying both the spin multiplicity and the symmetry, *nroot* is the number of roots, and *wgt(i)* is the weight for root *i* in the averaging over energies. The variable *nroot* can be written as a sum, *e.g.* 2+1 for a total of 3. This is convenient when adding asymptotic levels. The value of *wgt(i)* specified MUST have a decimal point, otherwise it is ignored. If the molecule is neutral, *ichrg* is omitted. If the molecule is a cation, *ichrg* is prefaced by a + sign, with + not followed by a number indicating +1. Anions are like cations, except - is used instead. If there is only one root, *nroot,(wgt(i),i=1,nroot)* can be omitted. Also, if dynamic weights are used, *(wgt(i),i=1,nroot)* can be omitted: any values read in will not be used.

5.3.1 term symbol

The term symbol can take on several forms. The simplest is *iSj*, with *i* the spin multiplicity and *j* the point group symmetry number. One can also replace *Sj* with the point group symmetry name, *e.g.* *A'* or *A''* for C_s symmetry. The recognized symmetry names are given next to the point group multiplication table in the output.

For linear molecules, higher symmetry can be indicated, *i.e.* instead of using *Sj*, one can use *Sig+*, *Sig-*, *Pi*, *Del*, *Phi*, or *Gam*, and for D_{2h} molecules, these are followed by *g* or *u*. For atoms, one can use *S*, *P*, *D*, or *F* for even parity states and for odd parity states, use *e.g.* *So*. The program will automatically ensure that the orbitals will have the proper symmetry. At this point, this only works if there are only σ and π (*s* and *p*) orbitals occupied for linear molecules (atoms).

For diatomic molecules, for *termsym* one can alternatively specify two atomic term symbols, *e.g.* *4So 4So* for the ground asymptote of N_2 . In this case, MPEC will generate all the diatomic terms that correlate to this asymptote.

5.4 dynamic weights

The keyword *dynw* sets the value of the dynamic weighting parameter(s). If this keyword is omitted, the weights will be fixed. Two different dynamic weighting functions can be selected. If only one dynamic weighting parameter is given, c_0 , we use the relative energy dynamic weight: $w^r(\Delta E^r) = \{2/[\exp(\Delta E^r) + \exp(-\Delta E^r)]\}^2$, with $\Delta E^r = (E - E_0)/c_0$. If two or three weighting parameters are given, c_0, c_1, c_2 , we use the absolute energy dynamic weight: $w^a(\Delta E^a) = 4/(c_2 + \exp(2\Delta E^a))$, with $\Delta E^a = (E - c_1)/c_0$. If c_2 is not given, it defaults to 2.5. The parameters c_0 and c_1 have units energy, and the effect of the damping can be seen in Fig. (1).

If only the value of the dynamic weighting parameter is given on the *dynw* line, the reference energy for computing the weights will be taken as the first root in the first symmetry in the states input. Alternatively, add the string *per* after the dynamic weighting parameter(s), and the reference energy will be the lowest energy for each symmetry. It is recommended that dynamic weighting not be used for calculations without a good initial guess for the orbitals. I have seen cases where the coupling with the dynamic weights keeps the solution from converging to the global minimum if one is starting far from the global minimum.

It should be noted that when the dynamic weights are turned on, the state averaged energy at a given iteration is not necessarily an upper bound to the final energy.

5.5 molden

The keyword *molden* causes the final orbitals to be written to the file *molden* using molden format.

5.6 rydberg

The keyword *rydberg* allows the optimization of the virtual orbitals. In this case, the *states* keyword should refer to N-1 electron states, *e.g.* if the system is neutral, *states* would specify the cation core to use. Then the Hamiltonian for one electron in a virtual state outside the N-1 electron state is constructed, and then averaged over the different N-1 electron states, using either fixed or dynamic weights, and diagonalized to yield optimized virtual orbitals. In the event that energies are less than the lowest energy N-1 electron state, the orbitals are flagged as Rydberg orbitals.

The *guess* keyword need not be supplied when *rydberg* is used.

Optionally, with this keyword one can specify the number of Rydberg orbitals for each symmetry. This is the abelian point group symmetry rather than cylindrical or spherical symmetry in cases where higher symmetry is used. This option is recommended if diabatic orbitals are to be generated, for the diabatic procedure will fail if the number of Rydberg orbitals changes with geometry.

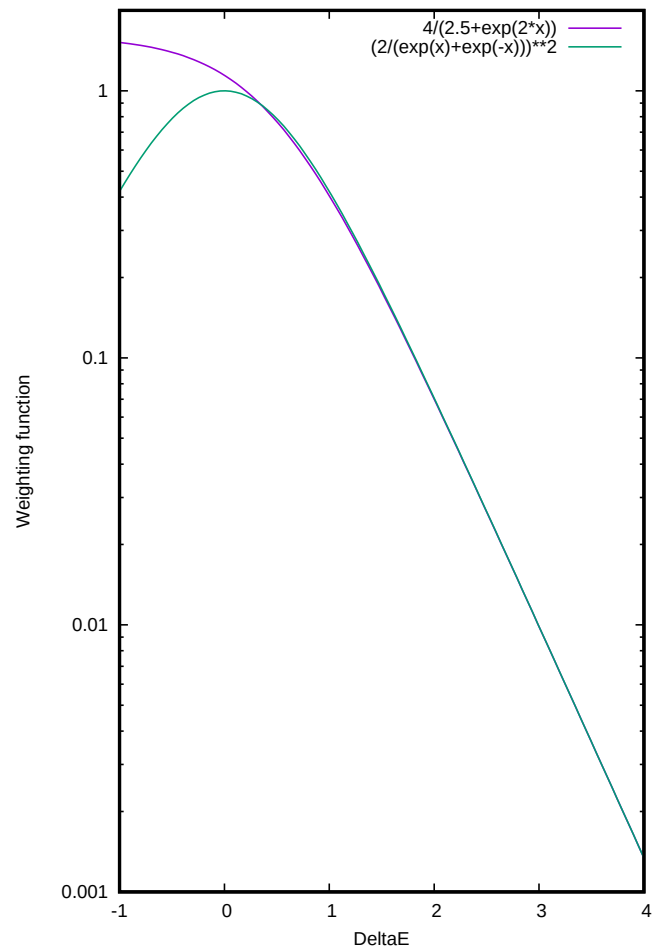


Figure 1: **Dynamic weighting function**

5.7 *grad*

Usage of this keyword turns on the analytic calculation of CAS gradients. This works for one or more electronic states as well as fixed or dynamic weights. Both non-relativistic and relativistic wavefunctions can be differentiated.

If this keyword had no argument, the totally symmetric orbital gradients will be computed. If it has the argument *bodc*, the Born-Openheimer diagonal correction will be computed. If more than one root is included in the calculation, the derivative coupling matrix elements are also computed.

The orbital gradient information is also saved in the orbital file *forbs*. When an orbital file with gradient information is used in an i²cMRCI-SD calculation, the gradients of the i²cMRCI-SD energies will also be computed. Warning: for i²cMRCI-SD energy gradients, you will want to use the *wconv* option to ensure that the wavefunction is sufficiently well converged that the wavefunction derivative terms that are neglected are sufficiently small.

If one uses *grade*, the state averaged energy gradient will be computed.

If one uses *min*, MPEC will optimize the geometry.

If one uses *stationary*, MPEC will search for the closest geometry where the norm of the gradient is zero.

5.8 miscellaneous keywords

sum causes a summary file to be written with the converged results to the file *summarycas*.

csf causes the FCI calculation to use a CSF basis rather than a determinant basis. Its argument specifies the maximum number of open shells kept in the now less than FCI diagonalization.

3rd allows one to modify the gradient threshold for switching from second order the third order orbital optimization.

epssym is the tolerance for the purity of orbital expectation values if spherical or cylindrical symmetry is used. The default value is 1d-9.

first specifies the number of macro iterations that go strictly downhill, *i.e.* ignore the Hessian in the orbital optimization.

wern allows one to specify the order of the approximation of the integrals during the micro iterations. Allowed arguments are 1, 2, or 3.

nowern forces the Augmented Hessian method to be used for the orbital optimization.

maxitmac sets the maximum number of macro iterations.

npdiag specifies the size of the P space matrix where we switch from a scalar to parallel diagonalization routine. The default is 500.

pthrs sets the energy threshold for partitioning the FCI Hamiltonian into a primary and secondary space.

maxps specifies the maximum size of the P space.

maxitci specifies the maximum number of Davidson iterations in the FCI diagonalization.

econv sets the energy convergence threshold

vconv sets the vector convergence threshold
gconv sets the gradient convergence threshold
nocan turns off the canonicalization of the orbitals
sneglect sets the tolerance to neglect near linear dependence in the one electron basis. This is based on the eigenvalues of the overlap matrix, so doesn't work properly when relativity is turned on. The default value is 1d-60.
usenormal controls the equations used in the DIIS extrapolations in the gradient calculations. The default is to use the normal equations in the least square fit. One can also use the singular-value-decomposition (svd) by using the argument *no*, *off*, or *svd*. The svd method is much more numerically stable, but also is much more expensive. One can also compare the results of the two methods by using the argument *test*.

6 DIAB

This program section allows one to generate diabatic orbitals.

The input consists of a series of lines, with each line containing the file name of the orbitals at a particular geometry. The first line will give the set of orbitals that will be made diabatic, while the subsequent lines give file names for orbitals at adjacent geometries which have previously been made diabatic. The new set of orbitals will be written to the file *forbs*.

One can also include the keyword *dtest* which allows the adjustment of the threshold for aborting the calculation if the product of the diagonals of the overlap matrix is less than the value of this parameter. The default value of *dtest* is 0.9.

In general, the orbitals fall into four classes: doubly occupied orbitals, active orbitals, Rydberg orbitals, and virtual orbitals. The orbitals in a particular class will be mixed together and rephased to maximize the similarity with the orbitals at adjacent geometries. If the orbitals can not be made sufficiently similar, the program will not write out the orbitals: set *dtest* above.

It should be noted that the number of orbitals in each class must be the same in all the files listed. The number of doubly occupied and active orbitals are specified in the CAS input. The number of Rydberg orbitals can either be automatically determined by MPEC or by user input with the *rydberg* keyword. The later choice is recommended if diabatic orbitals are to be generated.

7 CAT

This program section allows one to take orbital files from fragment calculations and concatenate them into a single file for a supermolecule calculation.

The input consists of a series of lines, with each line containing the file name of the orbitals of a fragment.

The concatenated orbitals are written to the file *forbs*.

The point group symmetries must be the same for each of the input orbital files, and all the files must either exclude special relativity or include special relativity.

8 MRCI

This program section controls the MRCI calculations.

If the *forbs* file is from a gradient calculation, the gradients of the i²cMRCI energy will be computed.

First a note on the output: as the CI iterations progress, more and more significant digits for the energies are printed out. The final digit of these energies is rounded, so the energies can appear to increase at times. This, however, is just an effect of the rounding and not an indication of non-variational energies. If multiple roots are calculated, when lower roots reach convergence, the letter

Table 4: MRCI quick reference

<i>0hole</i>	<i>1hole</i>	<i>2hole</i>	<i>2den</i>	<i>1fill</i>
<i>2fill</i>	<i>3fill</i>	<i>4fill</i>	<i>cache</i>	<i>cren</i>
<i>debug</i>	<i>decomp</i>	<i>doubo</i>	<i>doubx</i>	<i>dyn</i>
<i>econv</i>	<i>eiconv</i>	<i>itabort</i>	<i>maxit</i>	<i>maxiti</i>
<i>maxdiis</i>	<i>maxds</i>	<i>maxdsi</i>	<i>maxopens</i>	<i>maxps</i>
<i>maxrest</i>	<i>nointeract</i>	<i>norb</i>	<i>novguess</i>	<i>nowav</i>
<i>npdiag</i>	<i>pthrs</i>	<i>readc</i>	<i>ref</i>	<i>restart</i>
<i>save</i>	<i>sing</i>	<i>state</i>	<i>test</i>	<i>toldv</i>
<i>usenormal</i>	<i>wconv</i>	<i>xref</i>		

c is appended to the energy, and new trial vectors for these states will no longer be generated.

Besides the MRCI energy, the program also computes a hybrid Davidson's quadrupole correction. This is defined by the quantities E_{ref} and $|c_0|^2$. These two quantities are ambiguous in the multi-reference case. In the current version, we compute three different Davidson's corrections. The corrections $q(f)$ and $q(r)$ are computed using as E_{ref} the energy from diagonalizing the Hamiltonian defined by *idoubo* - *2fill*. We then for $q(f)$ we take c_0 to be the dot product of vector from the computation of E_{ref} with the final MRCI vector, and for $q(r)$, we take c_0 as the amplitude of the final MRCI vector in the Hilbert space from the computation of E_{ref} . The correction $q(sx)$ uses E_{ref} computed from the internal and singles part of the final vector, and c_0 is the fraction of the final vector that includes the internals and singles.

If the MRCI calculation follows a CAS calculation, most, if not all, of the input can be omitted.

8.1 occupancies

The keyword *doubo* is for the doubly occupied orbitals for each symmetry in all configurations. This defaults to *doub* from the CAS calculation from which the orbitals are taken.

The keyword *ref* is for the orbitals with occupancy in the reference configurations and excitations out of in the MRCI wavefunction. This defaults to *act* from the CAS calculation from which the orbitals are taken.

8.1.1 occupancy restrictions

The keyword *0hole* specifies a list of orbitals (*e.g.* *1s1 2s4* for reference orbital 1 symmetry 1 and reference orbital 2 symmetry 4) that are doubly occupied in all reference configurations but excitations are allowed out of them into the external space. Use this, for example, to correlate core electrons.

The keyword *1hole* specifies a list of orbitals that can have up to one hole in the reference configurations but are allowed excitations into the external space.

The keyword *2hole* specifies a list of orbitals that can have up to two holes in the reference configurations but are allowed excitations into the external space.

The keyword *1fill* specifies a list of orbitals that can have up to one electron in the reference configurations and excitations into the external space. Use this for example for Rydberg orbitals.

The keyword *2fill* specifies a list of orbitals that can have up to two electrons in the reference configurations and excitations into the external space.

The keyword *3fill* specifies a list of orbitals that can have up to three electrons in the reference configurations and excitations into the external space.

The keyword *4fill* specifies a list of orbitals that can have up to four electrons in the reference configurations and excitations into the external space.

8.2 state

The keyword *state* is followed by the arguments *ichrg*, *termsym*, *nroot*, with the same meaning as in the *states* keyword of the CAS code. If only one root is to be determined, *nroot* can be omitted. This defaults to the first state in the CAS calculation from which the orbitals are taken.

In addition, one can specify dynamic weighting parameters after this data. This dynamic weighting will be used to modify the convergence criteria for the roots, *e.g.* automatically allowing higher roots to be less well converged as energies split as a function of geometry.

8.3 excitations

The keyword *singx* controls the inclusion of single excitations. The argument of this keyword is either *off* or *on*. By default, *i.e.* *singx* omitted, *singx on* is assumed.

The keyword *doubx* controls the inclusion of double excitations. The arguments are *off*, *on*, or *uc*. The default, *i.e.* when *doubx* is omitted, is *doubx*

on. The argument *uc* means include the double excitations, but do not contract them.

8.4 internal contraction

The keyword *dynw* specifies the dynamic weighting parameter to use in generating the internal contraction. It uses the same weighting functions and input as in the CAS calculation.

The keyword *xref* allows one to use vectors of other symmetries in the contractions. This keyword is followed by a list of root and state specifications. The number of roots can be omitted if it is unity. The states are either specified by *Si*, or *i.e. Sig+*, *So*, *etc.* The number of roots is an integer following the state name, with a space or comma between them. This is sort of like the *state* specification, except the charge and spin multiplicity are not given: they are the same as the state for which the MRCI calculation will be carried out.

The keyword *ethred* allows one to change the default threshold for neglecting near linearly independent contraction vectors. The current default is -7, meaning the threshold is 10^{-7} .

The keyword *readc* causes the contraction coefficients to be generated from previously computed vectors. These come from a *wavef* file from a previous MRCI calculation. The argument of this keyword is the file name.

8.5 restarts

The MRCI program periodically saves restart information so that calculations can continue without losing information in case the calculation fails. This restart information is written to the file *wavef*. The restart information is written in a similar manner as the final wave function file, but not exactly the same. This means that a restart file can not accidentally be used with the property program.

The keyword *save* allows one to modify the frequency at which restart information is saved. The argument specifies how many iterations are skipped before restart information is saved.

The keyword *restart* allows one to restart the calculation. If there is no argument or the argument is *yes*, restart information will be read from the file *wavef*. If the argument is *no*, the calculation will not be restarted.

8.6 miscellaneous keywords

econv specifies the convergence criterion for energies. The default is 1D-9 Hartree.

eiconv specifies the convergence criterion for the internal space diagonalization, if there is more than one root. The default is 1D-9 Hartree. If there is only one root, *eiconv* starts out at 1d-3 and then is adjusted during the iterations so it becomes 10% of difference between the last energy and the energy from the current iteration.

npdiag specifies the size of the P space matrix where we switch from a scalar to parallel diagonalization routine. The default is 500.

pthrs specifies the relative energy threshold for including CSFs in the P space.

maxps specifies the maximum size of the P space.

maxit specifies the maximum number of iterations in the full diagonalization

maxiti specifies the maximum number of iterations in the diagonalization in the internal space

maxdiis specifies the maximum number of DIIS iterations before restart in the calculation of the derivatives of the internal contraction vectors.

maxds specifies the number iterations in the full diagonalization before contracting down to the best Davidson vectors.

maxdsi specifies the number iterations in the internal diagonalization before contracting down to the best Davidson vectors.

maxopens specifies the maximum number of open shells any CSF can have.

maxrest specifies the maximum number of restarts in the DIIS procedure to compute the derivatives of the internal contraction vectors.

nowave indicates that no *wavef* should be written.

decomp indicates that extra decomposition of the doubles contribution to the energy will be printed out.

norb causes natural orbitals to be computed. The argument should be either *average* to average the density over the roots computed before diagonalization or a root number to just use the density of that root.

cren indicates that the trial contraction vectors should be renormalized before they are orthogonalized. Using this will remove the dynamic weights from the generators of the contraction.

itabort is the iteration number to stop the calculations.

toldv is the convergence criterion for the derivative of the internal vectors. The default value is 1d-12.

usenormal controls the equations used in the DIIS extrapolations in the gradient calculations. The default is to use the normal equations in the least square fit. One can also use the singular-value-decomposition (svd) by using the argument *no*, *off*, or *svd*. The svd method is much more numerically stable, but also is much more expensive. One can also compare the results of the two methods by using the argument *test*.

wconv is the convergence criterion for the wavefunction. The default is to not use this option but rather converge the energy utilizing *econv*. This is a useful option when computing derivatives when one wants to ensure that the derivatives of the eigenvectors are indeed small enough to be neglected.

9 PROP

There are five possible keywords for this section, while all other lines of input are file names of wavefunctions used to compute the properties.

The keywords are *itype*, *genwf*, *sotm*, and *so4v*.

The keyword *itype* will have either the argument *ss*, *ssg*, or *ssb*. The argument *ss* means only compute Coulomb contributions to spin-orbit coupling, while *ssg* means include the Gaunt approximation to the Breit interaction, and *ssb* means include the full Breit interaction when computing spin-orbit matrix elements.

The keyword *genwf* has the argument *genwfeps*, the tolerance to check the generation of degenerate wavefunction components. If one gets the message “sz = *value* is too large in genwf”, where *value* is a floating point number, one can override this check by setting *genwfeps* to something larger than *value*.

The keyword *sotm* controls the computation of the spin-orbit contribution to transition moments. If there is no argument, the argument is *on*, or if this keyword is omitted, the spin-orbit contribution to the transition moments will be included. The argument *off* causes these contributions to be omitted.

The keyword *so4v* controls the computation of the four virtual contribution to the spin-orbit matrix elements. The argument is either *on* or *off*. Turning the four virtual integrals off can result in a significant reduction in CPU time with small loss of accuracy: see the He and N atom sample calculations.

There are two situations with the file names of the wavefunctions. In the first situation, the geometries used to generate the wavefunctions are all the same. Then properties will be computed. In the second situation, the geometries used to generate the wavefunctions are different. In this case, the wavefunction for the first file name will be made diabatic by comparing to the other wavefunctions included in the input. The new wavefunction will be saved to the file *dwfile* if the input file name is *wfile*.

If this section is included directly after a MRCI calculation, no input is required to obtain expectation values.

The program figures out what matrix elements to compute, depending on the type of molecule and whether or not special relativity is turned on, in which case spin-orbit matrix elements will be computed.

10 *debug*

Many program sections have the *debug* keyword. This causes extra debug print out. The allowed arguments for the *debug* keyword are given in the following table.

Table 5: debug arguments

<i>cannon</i>	<i>cas0</i>	<i>casopt</i>	<i>cont2ecsf</i>	<i>contractg</i>
<i>cupo2</i>	<i>dotdvecs</i>	<i>gencsf2</i>	<i>gencsf3</i>	<i>genergy</i>
<i>genfcn</i>	<i>genryd</i>	<i>grad</i>	<i>hccsfd</i>	<i>intcsf</i>
<i>makeguess</i>	<i>mofromao</i>	<i>mrci</i>	<i>opto</i>	<i>paraeri</i>
<i>parahf</i>	<i>parajkfromh</i>	<i>prntm2</i>	<i>psisopsi</i>	<i>restart</i>
<i>sortoutsym</i>	<i>stepwisecsf</i>	<i>updateg</i>	<i>updatex</i>	

11 Sample inputs

The sample runs given here have a two-fold purpose: the first being a test of all the features of the code, and the second being a demonstration on how one can use shell commands to perform loops and define variables so that duplicate inputs (and thus errors!) are avoided. The scripts given below are annotated with comments after the `#` character: both the shell and MPEC will ignore strings following `#`. Thus these scripts will run as annotated. These scripts are also given in the test directory in the distribution package as well as the outputs I generated on my computer. There are also `precis` files for ease in comparing the outputs from two runs. The command to run MPEC is called `runmpec` and defined by the script creating MPEC. See README file for more details.

The sample inputs must be run using the Borne shell, *i.e.* `sh runcontract.sample`.

When I run these on my laptop, I get pop-up windows asking for permission for MPEC to connect to the internet. MPEC does not connect to the internet, so either ignore these messages or click on the deny button. If your script runs a series of calculations, these pop-up windows will be created every time MPEC is run. An exception to this, I have found, is to click on the deny button for a pop-up window from the currently running calculation. Then there are no pop-up windows for subsequent calculations in the script.

11.1 C atom basis contraction

The script that runs this calculation is `runcontract.sample`.

In this example we generate relativistic contraction coefficients for the Carbon atom. We run non-relativistic calculations as well to double check the wavefunction we are using reproduces the tabulated contraction coefficients. The calculations using the dz basis start from orbitals that diagonalize the one-electron Hamiltonian while the subsequent basis sets use the orbitals from the previous basis as initial guesses.

```
. ~/.prempc
echo "C atom contraction example"
output=ccontr.sample`date -Idate`
/bin/rm $output
for shellrel in off ss          # loop over non-relativistic then relativistic
do
  shellguess=h0                # this creates a character string shell variable and sets its value to h0
  for shellbasis in dz tz qz 5z 6z # loop over basis sets
  do
    cat > input <<@
    &BG
    rel $shellrel               # $shellrel means substitute the contents of the shell variable shellrel
    C
    cc-pv$shellbasis u -dfghi   # uncontract basis, delete all except sp functions
    &CAS
    doub 2
    act 0 1 1 0 1
    guess $shellguess           # $shellguess means substitute the contents of the shell variable shellguess
    states
    3P
    @
    /bin/rm forbs               # delete forbs file if left over from another calculation
    runmpec < input >> $output
    mv forbs orbs               # rename final orbitals to orbs to be initial guess for next larger basis
    if [ $? -ne 0 ] ; then exit ; fi # if rename fails, bail
    shellguess=readin           # reset shellguess variable to read from orbs file next time around
  done                          # end of basis set loop
done                            # end of relativistic loop
```

So, for example, the input for the cc-pvdz basis s functions is

```
#BASIS SET: (9s,4p,1d) -> [3s,2p,1d]
C      S
6665.0000000  0.0006920 -0.0001460
1000.0000000  0.0053290 -0.0011540
228.0000000   0.0270770 -0.0057250
64.7100000   0.1017180 -0.0233120
21.0600000   0.2747400 -0.0639550
7.4950000    0.4485640 -0.1499810
2.7970000    0.2850740 -0.1272620
0.5215000    0.0152040  0.5445290
```

while the converged ao basis orbitals for the non-relativistic calculation are

```
1  6.921597E-04 -1.461361E-04 ...
2  5.329013E-03 -1.154338E-03 ...
3  2.707689E-02 -5.725498E-03 ...
4  1.017182E-01 -2.331215E-02 ...
5  2.747402E-01 -6.395540E-02 ...
6  4.485638E-01 -1.499811E-01 ...
7  2.850744E-01 -1.272622E-01 ...
8  1.520382E-02  5.445296E-01 ...
.
.
.
```

which are the same within the number of significant figures given.

To add relativistic contraction coefficients we add the code “rc” at the end of the line and simply cut and paste the converged ao basis orbitals from the relativistic calculation to get

```
#BASIS SET: (9s,4p,1d) -> [3s,2p,1d]
C      S
6665.0000000  0.0006920 -0.0001460 rc  7.105961E-04  6.946040E-04 -1.502102E-04 -1.467133E-04
1000.0000000  0.0053290 -0.0011540 rc  5.382788E-03  5.343683E-03 -1.167157E-03 -1.158528E-03
228.0000000   0.0270770 -0.0057250 rc  2.720712E-02  2.712833E-02 -5.758726E-03 -5.739008E-03
64.7100000   0.1017180 -0.0233120 rc  1.019287E-01  1.018329E-01 -2.338580E-02 -2.335872E-02
21.0600000   0.2747400 -0.0639550 rc  2.748872E-01  2.748390E-01 -6.402886E-02 -6.399401E-02
7.4950000    0.4485640 -0.1499810 rc  4.483559E-01  4.484160E-01 -1.500884E-01 -1.500704E-01
2.7970000    0.2850740 -0.1272620 rc  2.846983E-01  2.847716E-01 -1.268257E-01 -1.268091E-01
0.5215000    0.0152040  0.5445290 rc  1.517080E-02  1.517560E-02  5.452046E-01  5.452149E-01
```

when the converged ao basis orbitals are printed out as

```
m-n      1k1      1k2      2k1      2k2      3k1
1  7.105961E-04  6.946040E-04 -1.502102E-04 -1.467134E-04 ...
2  5.382788E-03  5.343683E-03 -1.167157E-03 -1.158528E-03 ...
3  2.720712E-02  2.712833E-02 -5.758726E-03 -5.739008E-03 ...
4  1.019287E-01  1.018329E-01 -2.338580E-02 -2.335872E-02 ...
5  2.748872E-01  2.748390E-01 -6.402886E-02 -6.399402E-02 ...
6  4.483559E-01  4.484160E-01 -1.500884E-01 -1.500704E-01 ...
7  2.846983E-01  2.847716E-01 -1.268257E-01 -1.268091E-01 ...
8  1.517080E-02  1.517560E-02  5.452046E-01  5.452149E-01 ...
.
.
.
```

11.2 Ne atom

The script that runs this calculation is runne.sample.

This example tests the Dirac-Hartree-Fock program and compares to the equivalent results using the Spin-Free formalism. We also take the basis set directly from the input. We also compute the Born-Oppenheimer diagonal correction by analytically differentiating the orbitals.

```
. ~/.prempc
echo "Ne atom relativity sample"
output=ne.sample'date -Idate'
/bin/rm $output
cat > input <<@
&BG
rel dhf ssssB          # Dirac-Hartree-Fock using all integrals and computing Breit interaction
Ne
0. 0. 0.              # normally one does not need the geometry for the atom, but since
```

```

@
cat Ne.inp >> input          # we are reading in the basis, we need it.
cat >> input <<@             # copy the basis input to the middle of the mpec input
&SOHF
@
/bin/rm forbs                # delete any left over orbitals
runmpec < input >> $output
if [ $? -ne 0 ] ; then exit ; fi
cat > input <<@
&BG
rel ssss
Ne
0. 0. 0.
@
cat Ne.inp >> input
cat >> input <<@
&CAS
doub 2
act 0 1 1 0 1
guess
grad bodc
states
1S
&MRCI                        # state defaults to first CAS state
singx off                    # turn off single excitations
doubx off                    # turn off double excitations
&PROP
itype ssg                    # turn on Gaunt
@
/bin/rm forbs                # delete any left over orbitals
runmpec < input >> $output
cat summary > /dev/null
if [ $? -ne 0 ] ; then exit ; fi
cat > input <<@
&PROP
itype ssb                    # turn on Breit
wavef                        # since this not part of the MRCI run, we need to specify wavefunction file
@
runmpec < input >> $output
grep '>' $output > $output.precis # make summary file

```

This same calculation was run using Harry Quiney's code Bertha, and from that code we obtain -128.691923 for the DHF energy and 1.6644E-002 for the Breit correction. MPEC gives -128.691921 for the DHF energy, 1.7542E-02 for the Gaunt approximation to the Breit correction and 1.6644E-02 for the Breit correction. The calculations agree within $1 \mu E_h$. In comparison, the Spin-Free HF energy is -128.6919209, and for this wavefunction the Gaunt approximation to the Breit interaction is 1.7544e-02 and the Breit interaction is 1.6645E-02.

The Born-Openheimer diagonal correction (bodc) computing by differentiating the CAS wavefunction is 114.64373 divided by the nuclear mass. This can be compared to the mass polarization (mp) correction with out mass computed by the property code:

```
< 1 1S|mp w/o mass| 1 1S>= 1.1464370E+02
```

which is essentially exactly the same.

11.3 He atom

The script that runs this calculation is runhe.sample.

In this example we compute an atomic line list for low lying states of He, including the $^3P^o$ state. The spin-orbit splitting of this state is what motivated Breit to develop his interaction. We perform a MCHF calculation to obtain valance orbitals, then use the Rydberg option to generate the low lying Rydberg orbitals. Then we loop over electronic states to generate the wavefunction files which are then fed into the property program. We test including or excluding the spin-orbit contribution to the transition moments, what spin-orbit splitting

we get from excluding or including the Breit interaction, and how this changes if we neglect the four virtual integrals.

```
. ~/.premprec # execute script to define environment
echo "He atom sample"
output=he.sample' date -Idate '
/bin/rm $output
shellhebasis="cc-pvtz &aug +2sp" # create a shell variable as a shorthand for basis set
cat > input <<@
&BG # optimize valence orbitals
rel
He
$shellhebasis # $shellhebasis means substitute the contents of the shell variable shellhebasis
&CAS
guess
states
1S
@
/bin/rm forbs # delete any left over orbitals
runmprec < input >> $output
mv forbs orbs # rename to orbs to be initial guess for Rydberg run
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
cat > input <<@
&BG
rel
He
$shellhebasis
&CAS
ryd # optimize virtual orbitals
states # need to specify cation state here
+ 2S
@
runmprec < input >> $output
cp forbs /dev/null # dummy copy to check status
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
/bin/rm prop.in summary.he # clean slate
for shellstate in 1S,2 3S 3Po 1Po # loop over electronic states: note we are getting 2 1S states
do
wf=wavef$shellstate # this will be the name of the wavefunction file
echo $wf >> prop.in # build up list of wavefunction file names
cat > input <<@
&MRCI
refo 2 1 1 0 1 # rydberg orbitals are in reference space
state $shellstate
@
/bin/rm summary wavef # delete left overs
runmprec < input >> $output
cat summary >> summary.he # make list of results
if [ $? -ne 0 ] ; then exit ; fi # summary is only written if calculation runs properly
mv wavef $wf # rename wavef file
done # end of loop over electronic states
for shellso in off on # loop over spin-orbit transition moments on or off
do
cat > input <<@
&PROP # start of property input
itype ss
sotm $shellso
@
cat prop.in >> input # add list of wavefunction files
runmprec < input >> $output
done # end loop over spin-orbit transition moments
for shellv4 in on off # loop over including or neglecting 4 virtual integrals
do
for shelltype in ssg ssb # loop over Gaunt and Breit interaction
do
cat > input <<@
&PROP # start of property input
itype $shelltype
so4v $shellv4
@
cat prop.in >> input # add list of wavefunction files
runmprec < input >> $output
done # end of Gaunt/Breit loop
done # end of 4 virt loop
grep '>' $output > $output.precis # tabulate results to compare
```

The summary file from this calculation is

avg MCHF energy	no. double excitations	state	CI Energies
-2.8613158E+00	75	1S	-2.90073850304 -2.143996770
-2.8613158E+00	49	3S	-2.1738350422
-2.8613158E+00	48	3Po	-2.1316382061
-2.8613158E+00	48	1Po	-2.122116281

It should be noted that the energy for the $^1P^o$ state is printed out to one less significant figure than the other states. This is because the energy for this state did not converge as well as the other states, so MPEC modified the number of significant figures given. These energies are rounded rather than truncated.

The inclusion of the spin-orbit transition moments increases the number of lines in the line list from 17 to 19, and in particular adds a 3S to 1S transition in the VUV.

We parameterize the spin-orbital splitting as a coefficient A_q that multiplies a product of Clebsch-Gordan coefficients. With just Coulomb spin-orbit, we only have $q = 1$, while for the Breit interaction and the Gaunt approximation to the Breit interaction, we also have $q = 0$ and $q = 2$. For the Coulomb spin-orbit, we have

```
A1  1 3Po      1 3Po      >=  1.803071E-06
```

while for the Gaunt approximation to the Breit interaction we obtain

```

              4v on      4v off
A0  1 3Po      1 3Po      >=  2.489745E-06  2.456938E-06
A1  1 3Po      1 3Po      >= -1.930869E-06 -1.902219E-06
A2  1 3Po      1 3Po      >=  2.435852E-06  2.364902E-06
```

and for the full Breit interaction

```

              4v on      4v off
A0  1 3Po      1 3Po      >=  1.937282E-06  1.913255E-06
A1  1 3Po      1 3Po      >= -1.930854E-06 -1.902204E-06
A2  1 3Po      1 3Po      >=  2.435852E-06  2.424310E-06
```

While differences between the Gaunt approximation and the full Breit interaction are small, the difference with the Coulomb spin-orbit is very significant, for the sign on A_1 changes. With the Coulomb spin-orbit interaction, $J = 0$ is the lowest multiplet level, while with the Breit interaction, $J = 2$ is the lowest multiplet level. In particular, from the full Breit calculation, we find that the average difference between our results and the NIST results for the $^3P^o$ multiplet is 336.031 cm^{-1} with a root-mean-square error of only 0.022 cm^{-1} . If we neglect the four virtual integrals, for the same quantities we get 335.429 cm^{-1} and 0.016 cm^{-1} , essentially the same result. Needless to say, there is a huge decrease in computer time when the four-virtual integrals are neglected.

11.4 N atom

The script that runs this calculation is `runn.sample`.

In this test we make sure everything works for an odd number of electrons. Of particular interest to NASA is the 174nm line cluster arising from the 2P Rydberg state to the $^2P^o$ metastable state.

```

. ~/.premprec          # set up mpec environment
echo "N atom sample"
shellnbasis="cc-pvtz &aug +2s" # create shell variable with basis description in it
output=n.sample'date -ldate'
/bin/rm $output
cat > input <<@
&BG
rel
N
$shellnbasis           # substitute contents of shell variable shellnbasis
&CAS
guess
states                # valence states
```

```

4So
2Do
2Po
4P
@
/bin/rm forbs # delete left over orbitals
runmpc < input >> $output
mv forbs orbs # copy orbs to be initial guess for next calculation
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cat > input <<@ # optimize Rydberg orbitals
&BG
rel
N
$shellnbasis
&CAS
ryd
states
+ 3P # N+ core
@
runmpc < input >> $output # don't do $? test on this run, for mrci calculations with fail without forbs
/bin/rm prop.in summary.N # delete left overs
for shellstate in 4So 2Do 2Po 4P,2 2P # loop over low lying electronic states
do
wf=wavef$shellstate
/bin/rm $wf
echo $wf >> prop.in # build up PROP input
cat > input <<@
&MRCI
doub0 1
refo 2 1 1 0 1 # include s Rydberg orbital in reference space
ifill 2s1 # but only allow a maximum of 1 electron in it
state $shellstate # grab state name from shell variable shellstate
@
/bin/rm summary wavef # delete left overs
runmpc < input >> $output
cat summary >> summary.N
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed. We must test on summary rather than wavef, for wavef contains
mv wavef $wf # while summary is only written when calculation completes
done # end loop over electronic states
for shellso in on off # check on the importance of spin-orbit transition moments
do
cat > input <<@ # first part of PROP input
&PROP
itype ss
sotm $shellso
@
cat prop.in >> input # rest of PROP input
runmpc < input >> $output
done # end spin-orbital transition moment loop
for shellv4 in on off # loop over including or excluding 4 virt integrals
do
for shelltype in ssg ssb # loop over Gaunt or full Breit
do
cat > input <<@ # first part of PROP input
&PROP
itype $shelltype
so4v $shellv4
@
cat prop.in >> input # rest of PROP input
runmpc < input >> $output
done # end Gaunt/Breit loop
done # end of 4 virt loop
grep '' $output > $output.precis

```

The summary file for this run is

avg MCHF energy	no. double state excitations	CI Energies	
-5.4246852E+01	2660 4So	-54.5447180938	
-5.4246852E+01	4188 2Do	-54.454505290	
-5.4246852E+01	5193 2Po	-54.410548686	
-5.4246852E+01	5658 4P	-54.16764711772	-54.1409600057
-5.4246852E+01	5467 2P	-54.1529227135	

Like in the He atom example, these energies are given to different numbers of significant figures. This reflects how well each energy is converged, and the last digit has been rounded rather than just truncating the results.

We see in this example that including the spin-orbit transition moments only adds one addition line and only has significant effect (i.e. more than 1 per cent) on a hand full of lines.

For the $2P^o \leftarrow 2P$ transition, the Coulomb spin-orbit splitting of the upper state is 101 cm^{-1} , and the

			Coulomb	Coulomb-Gaunt	Coulomb-Breit
2P	1	1/2	-54.15119041 86369.34	-54.14636064 87429.36	-54.14648132 87402.87
2P	1	3/2	-54.15073047 86470.29	-54.14600919 87506.49	-54.14612989 87480.00

Gaunt and Breit splittings are both 77 cm^{-1} , compared to the experimental splitting of 83 cm^{-1} . For the lower state, the Coulomb

			Coulomb	Coulomb-Gaunt	Coulomb-Breit
2Po	1	3/2	-54.40851073 29894.06	-54.40371274 30947.10	-54.40383914 30919.36
2Po	1	1/2	-54.40851855 29892.34	-54.40371024 30947.65	-54.40383664 30919.90

spin-orbit splitting is 1.7 cm^{-1} , and the Gaunt and Breit splittings are both -0.6 cm^{-1} , compared to the experimental splitting of 0.39 cm^{-1} . If we neglect the four-virtual integrals, the Breit splitting for the lower state is -0.5 cm^{-1} and for the upper state it is 77 cm^{-1} .

The spectral lines for the transition of interest predicted using Coulomb spin-orbit are

	transition e/wl		Einstein A	lower		upper				
	1/cm	nm	1/s	rt	j	rt	j	type	elow(1/cm)	
25	5.6577946E+04	1.7674731E+02	2.2494426E+07	1	2Po	1/2 <-	1	2P	3/2 e1	2.9892345E+04
26	5.6576230E+04	1.7675268E+02	1.1340122E+08	1	2Po	3/2 <-	1	2P	3/2 e1	2.9894061E+04
27	5.6476999E+04	1.7706323E+02	8.9504400E+07	1	2Po	1/2 <-	1	2P	1/2 e1	2.9892345E+04
28	5.6475283E+04	1.7706861E+02	4.2898192E+07	1	2Po	3/2 <-	1	2P	1/2 e1	2.9894061E+04

while Coulomb-Gaunt spin-orbit gives

25	5.6559393E+04	1.7680529E+02	1.1316585E+08	1	2Po	3/2 <-	1 2P	3/2 e1	3.0947099E+04
26	5.6558845E+04	1.7680700E+02	2.2472844E+07	1	2Po	1/2 <-	1 2P	3/2 e1	3.0947646E+04
27	5.6482257E+04	1.7704675E+02	4.3188652E+07	1	2Po	3/2 <-	1 2P	1/2 e1	3.0947099E+04
28	5.6481709E+04	1.7704847E+02	8.9527920E+07	1	2Po	1/2 <-	1 2P	1/2 e1	3.0947646E+04

as compared to the experimental values

	Einstein A				
nm	1/s	lower	upper		
174.2719	2.12e+07	2Po 1/2 <-	2P 3/2		
174.2731	1.05e+08	2Po 3/2 <-	2P 3/2		
174.5249	8.35e+07	2Po 1/2 <-	2P 1/2		
174.5260	4.01e+07	2Po 3/2 <-	2P 1/2		

11.5 HF molecule

The script that runs this calculation is runhf.sample.

In this test, we compute the four lowest potential curves of $1\Sigma^+$ symmetry. This is a particularly challenging problem since the first excited state is nearly triply degenerate, with two fold degeneracy from the interaction between the hydrogen $n=2$ excited state, and an accidental degeneracy with the F^- ground state. Thus we first optimize valence orbitals without extra diffuse functions on H, then we add the extra diffuse functions to compute the Rydberg orbitals. Note we specify the number of Rydberg orbitals per symmetry: this is important because if we do not do this, the number of orbitals flagged as Rydberg orbitals can change with geometry, which will cause the diabatic orbital procedure to fail. We also do some extra CAS calculations for the first geometry to get good initial guess orbitals. For other than the first geometry, we compute diabatic orbitals and wavefunctions: the diabatic and adiabatic solutions are co-incident for the first geometry. In the ic^2MRCI calculations, we correlate the F 2s electrons,

even though they were kept doubly occupied in the CAS calculations, and we only allow at maximum one electron in the Rydberg orbitals when defining the reference wavefunction. Finally, we run both non-relativistic and relativistic calculations, and for the latter, we compute the Gaunt correction and other properties.

```
. ~/.premprec
echo "HF molecule sample"
fbasis="cc-pvtz &aug +sp"          # create shell variable with F basis information
hbasis="cc-pvtz &aug"              # create shell variable with H basis information
shellirel=0
irel=0
for shellrel in off ss              # loop over relativity off or on
do
/bin/rm summary.ldt$shellrel        # clean slate
shellpass=0                        # when shellpass=0, we will be carrying out extra calculations for initial guess
for shellr in 1000. 100. 50. 25. 10. # loop over geometries
do
echo "for bond length" $shellr
output=hfout`date -Idate`$shellrel$shellr
/bin/rm $output
if [ $shellpass -eq 0 ]             # for initial geometry
then
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
F is b
$fbasis
&CAS
doub 2                             # doubly occupy F 2s
act 1 1 1 0
guess                              # generate starting orbitals from one-electron Hamiltonian
states
+ 2Sig+                             # we are going for F, i.e. HF+ to get initial orbitals
+ 2Pi                               # we could have also used 2Po 1S to specify states
@
/bin/rm forbs
runmprec < input >> $output         # delete any orbitals lying about
mv forbs orbs                      # rename to orbs to act as initial guess
if [ $? -ne 0 ] ; then exit ; fi    # bail if calculation failed
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
F is b
$fbasis
&CAS
doub 2                             # F 2s doubly occupied
act 2 1 1 0                        # add H 1s to active space
states                             # note no guess line because default is to read in a guess
2Po 2S                             # state average over all states correlating to F 2Po + H 2S
@
runmprec < input >> $output
mv forbs orbs                      # rename so guess for next calculation
if [ $? -ne 0 ] ; then exit ; fi    # bail if calculation failed
fi
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
F is b
$fbasis
&CAS
dynw 0.4 -99.518                   # use absolute energy dynamic weight: -99.53 is roughly the energy of F-
doub 2                             # doubly occupy F 2s
act 3 2 2 0                        # augment active space with p shell to better describe F-
states                             # again no guess line required
2Po 2S                             # states correlating to F 2Po + H 2S
1S 1S 5.                           # states correlating to F- 1S boost weight this
@
runmprec < input >> $output
mv forbs orbs                      # rename so guess for next calculation
```

```

if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cp orbs orbsv # make copy for initial guess for next geometry
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis +2s +3p # now augment H basis to go for Rydberg orbitals
F is b
$fbasis
&CAS
dynw 0.1 -99.53 # use absolute energy dynamic weight
doub 2
act 3 2 2 0
ryd 2 1 1 0 # specify number of rydberg orbitals of each symmetry
states
+ 2Po 1S # core: states correlating to 2Po F
@
runmpec < input >> $output
cp forbs orbs # make copy for diabatic orbital calculation
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
if [ $shellpass -eq 1 ] # this is not first geometry, so make diabatic
then
cat > input <<@
&DIAB
orbs
$shellref # orbital file for last geometry
@
/bin/rm forbs # delete from last calculation
runmpec < input >> $output
cp forbs /dev/null # this is a dummy copy so we can examine return code
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
fi # end of diabatic block
mv forbs forbsr$shellrel$shellr # save final orbs
mv orbsv orbs # copy saved smaller basis orbitals for initial guess at next geometry
shellref=forbsr$shellrel$shellr # assign file for diabatic transformation at next geometry
cp $shellref forbs # mrci code runs off of forbs
/bin/rm summary wavef # delete left overs
cat > input <<@
&MRCI
doub 1 # F 1s doubly occupied
pthrs 5. # internal ci sometimes has trouble converging, so give it some help
refo 6 3 3 0 # we now include F 2s as well as H Rydberg orbitals
Ohole 1s1 # since F 2s was doubly occupied in CAS, we want to make same restriction
ifill 5s1 6s1 3s2 3s3 # put at most 1 electron in H Rydberg orbitals in reference
dynw 0.4 -99.584 # dynamically weight internal contraction: -99.584 is energy of F- with this reference space
state 1Sig+ 5 # go after 5 roots
@
runmpec < input >> $output
cat summary >> summary.hf$shellrel # build up summary file
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
if [ $shellpass -eq 1 ] # if we are not first geometry, form diabatic wavefunctions
then
cat > input <<@
&PROP
wavef # current geometry
$shellrefwf # last geometry
@
runmpec < input >> $output
mv dwavef wavef # rename to wavef so we can use same code for first geometry
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
fi # end of diabatic wavefunction block
mv wavef wavef$shellrel$shellr # save for later
shellrefwf=wavef$shellrel$shellr # set file name for next geometry
shellpass=1 # reset shellpass
if [ $shellrel -eq 1 ] # if relativistic calculation, compute Gaunt shift
then
for shellv4 in on off # loop over including or neglecting 4 virtual integrals
do
cat > input <<@
&PROP
itype ssg
so4v $shellv4
$shellrefwf
@
runmpec < input >> $output
done # end loop over 4 virt integrals
fi
grep '>' $output > $output.precis
done # end of loop over geometries.
shellrel=1 # reset for relativistic calculations
done # end of loop over relativity

```

The summary file for the non-relativistic case is

Bond Length	avg MCHF energy	no. double State excitations	CI Energies	
1.0000000E+03	-9.9792274E+01	172668 1Sig+	-100.1231992869	-99.7482349763
-99.7480183762	-99.7479449450	-99.742941143		
1.0000000E+02	-9.9794703E+01	172668 1Sig+	-100.1231905358	-99.751964224
-99.7482262253	-99.7480096303	-99.7479360563		
5.0000000E+01	-9.9797503E+01	172668 1Sig+	-100.1231811487	-99.761989464
-99.7482168644	-99.7480004023	-99.7479265418		
2.5000000E+01	-9.9803401E+01	172668 1Sig+	-100.1231645494	-99.782048081
-99.7482036481	-99.7479882849	-99.7479111771		
1.0000000E+01	-9.9823266E+01	173926 1Sig+	-100.12314397552	-99.8430375985
-99.74801796884	-99.74766786253	-99.744923086		

while the one for the relativistic case is

1.0000000E+03	-9.9875873E+01	172668 1Sig+	-100.2151736135	-99.8402048404
-99.8399871333	-99.8399128086	-99.834776348		
1.0000000E+02	-9.9878818E+01	172668 1Sig+	-100.2151673065	-99.843791673
-99.8401985334	-99.8399808313	-99.8399063968		
5.0000000E+01	-9.9882163E+01	172668 1Sig+	-100.2151605970	-99.853808631
-99.8401918497	-99.8399742807	-99.8398995882		
2.5000000E+01	-9.9889072E+01	172668 1Sig+	-100.2151492170	-99.873851776
-99.8401838448	-99.8399673889	-99.8398894511		
1.0000000E+01	-9.9911379E+01	173926 1Sig+	-100.21514337312	-99.9348021943
-99.8400094477	-99.83966107689	-99.836886936		

11.6 OH molecule

The script that runs this calculation is runoh.sample.

This is a rather contrived example. It proceeds very much like the HF example, but with one rather important difference: I now use a core-valence basis set. In this case, both electron and positron solutions are available to the basis, so the program carries out the MCHF optimization using as a basis the electron solutions from diagonalizing the one electron Hamiltonian. This requires extra manipulation to ensure one obtains a good guess when using a different basis or geometry that does not introduce positron solutions. As this is a contrived example, it does not run to completion, for the program fails to form diabatic orbitals at one geometry. The script stops when that happens.

```
. ~/.premprec
obasis="cc-pvtz &dwscv &aug +sp" # shorthand for O basis
hbasis="cc-pvtz &aug" # shorthand for H basis
for shellrel in ss
do
/bin/rm summary.oh$shellrel
shellpass=0 # when shellpass=0, we will be carrying out extra calculations for initial guess
for shellr in 100. 50. 25. 10. 8. # loop over geometries
do
echo "for bond length" $shellr
output=ohout`date -Idate`$shellrel$shellr
/bin/rm $output
if [ $shellpass -eq 0 ] # for initial geometry
then
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
O is b
$obasis
&CAS
doub 2 # doubly occupy O 2s
act 1 1 1 0
guess # generate starting orbitals from one-electron Hamiltonian
states
+ 3P 1S # we are going for O, i.e. OH+ to get initial orbitals
@
/bin/rm forbs # delete any orbitals lying about
runmprec < input >> $output
mv forbs orbs # rename to orbs to act as initial guess
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cat > input <<@
```

```

&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
O is b
$obasis
&CAS
doub 2
act 2 1 1 0
states
3P 2S
@
runmpec < input >> $output
mv forbs orbs
if [ $? -ne 0 ] ; then exit ; fi
fi
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis
O is b
$obasis
&CAS
dynw 0.1 -75.32
doub 2
act 3 2 2 0
states
3P 2S
2Po 1S
@
runmpec < input >> $output
mv forbs orbs
if [ $? -ne 0 ] ; then exit ; fi
cp orbs orbsv
cat > input <<@
&BG
rel $shellrel
ends(1)=a to b
rs $shellr
H is a
$hbasis +2sp
O is b
$obasis
&CAS
dynw 0.1 -75.32
doub 2
act 3 2 2 0
ryd 2 1 1 0
states
+ 3P 1S
@
runmpec < input >> $output
cp forbs orbs
if [ $? -ne 0 ] ; then exit ; fi
if [ $shellpass -eq 1 ]
then
cat > input <<@
&DIAB
orbs
$shellref
@
/bin/rm forbs
runmpec < input >> $output
cp forbs orbs
if [ $? -ne 0 ] ; then exit ; fi
fi
mv forbs forbs$shellrel$shellr
mv orbsv orbs
shellref=forbs$shellrel$shellr
shellpass=1
done
done
# 0 2s doubly occupied
# add H 1s to active space
# note no guess line because default is to read in a guess
# state average over all states correlating to 0 3P + H 2S
# rename so guess for next calculation
# bail if calculation failed
# ok, we are now done with preliminary guess. This will only happen once.
# use absolute energy dynamic weight: -75.32 is roughly the energy of 0-
# doubly occupy 0 2s
# augment active space with p shell to better describe 0-
# again no guess line required
# states correlating to 0 3P + H 2S
# states correlating to 0-
# rename so guess for next calculation
# bail if calculation failed
# make copy for initial guess for next geometry
# now augment H basis to go for Rydberg orbitals
# use absolute energy dynamic weight
# specify number of rydberg orbitals of each symmetry
# cation core: states correlating to 3P 0
# make copy for diabatic orbital calculation
# bail of calculation failed
# this is not first geometry, so make diabatic
# orbital file for last geometry
# delete from last calculation
# this is a dummy copy so we can examine return code
# bail if calculation failed
# end of diabatic block
# save final orbs
# copy saved smaller basis orbitals for initial guess at next geometry
# assign file for diabatic transformation at next geometry
# reset shellpass
# end of loop over geometries.
# end of loop over relativity

```

11.7 NO⁺ molecule

The script that runs this calculation is runno+.sample.

In this test we illustrate a problem with internal contraction for ions, and how we can solve it with MPEC. The problem occurs in the asymptotic region and the interplay between the $N^+ 4S^o$ and $O^+ 4S^o$ asymptote versus the $N^+ 3P$ and $O^+ 3P$ asymptote. In an accurate calculation, the $N+O^+$ asymptotic is lower in energy, however due to higher degeneracies, at the CAS level, the N^++O asymptote is lower. If one bases the contraction coefficients on the vectors computed from the reference space, one needs to include multiple roots to access the $N+O^+$ asymptote: if one only one root is included, the ic^2MRCI calculation converges to the wrong asymptote. However, if one first carries out an calculation including only single excitations, and uses that wavefunction to generate the contraction coefficients, one can obtain a good global solution from a single root calculation. We run four calculations per geometry: uncontracted MRCI for reference, ic^2MRCI for a single root, MRCI using only single excitations, and finally the ic^2MRCI calculation with the MRCI-S vectors to generate the contraction coefficients.

```
. ~/.prempc # set up mpec environment
echo "NO+ sample"
nbasis="cc-pvtz &aug" # shorthand for N basis
obasis="cc-pvtz &aug" # shorthand for O basis
shellguess=h0
/bin/rm summary.no+.uc summary.no+.1r summary.no+.s # start with clean slate
for shellr in 1000. 100. 50. 25. 10. 8. 6. 4. 3. 2. # loop over geometries
do
echo "for bond length" $shellr
output=no+out`date -Idate`$shellr
/bin/rm $output
cat > input <<@
&BG
ends(1)=a to b
r $shellr
N is a
$nbasis
O is b
$obasis
&CAS
doub 3 # doubly occupy O 2s
act 3 2 2 0
guess $shellguess # generate starting orbitals from one-electron Hamiltonian
states
+ 4So 4So # 4So N and 4So O+
+ 3P 3P # 3P N+ and 3P O
@
/bin/rm forbs # delete any orbitals lying about
runmpc < input >> $output
cp forbs orbs # copy to orbs to act as initial guess
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cat > input <<@
&MRCI
doub 2 # correlate Oxygen 2s
refo 4 2 2 0 # thus 2s is in reference space
Ohole 1s1 # but kept doubly occupied in reference wavefunction
state # oops, forgot and didn't put term symbol on same line ... (mpec didn't care)
+ 1Sig+ 3 0.001 # only worry about convergence of lowest root
doubx u # uncontract doubles
&PROP
@
/bin/rm summary
runmpc < input >> $output
mv wavef wavefuc$shellr
cat summary >> summary.no+.uc
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cat > input <<@
&MRCI
doub 2
refo 4 2 2 0
Ohole 1s1
state + 1Sig+ # naively go after just one root ... and get wrong answer
doubx # since converged to N+ and O, dipole has wrong sign
&PROP
@
/bin/rm summary # delete left overs
runmpc < input >> $output
cat summary >> summary.no+.1r
```

```

if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
cat > input <<@
&MRCI
doubi 2
refo 4 2 2 0
Ohole 1s1
state + 1Sig+ 3 # need 3 roots to ensure convergence to correct ground state
doubx off # just allow single excitations
@
/bin/rm summary # delete left overs
runmpc < input >> $output
cp summary /dev/null # dummy copy so we can check return code
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
mv wavef wavefmrctis$shellr
cat > input <<@
&MRCI
doubi 2
refo 4 2 2 0
Ohole 1s1
state + 1Sig+
readc wavefmrctis$shellr # read vectors for contraction coefficients from mrcis calculation
&PROP # dipole will now have correct sign
@
/bin/rm summary
runmpc < input >> $output
cat summary >> summary.no+.s
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
shellguess=readin # reset guess to readin orbitals
grep '>' $output > $output.precis
done # end of loop over geometries.

```

There are several ways one can tell the difference between $N^+ O$ and $N O^+$: the first is the dipole moment: since the origin of our coordinate system is at the nuclear center of mass, the dipole moment of $N^+ O$ will be the value of the N z coordinate, *i.e.* -533.19953280, for a bond length of 1000 a_o , and the dipole moment of $N O^+$ will be the value of the O z coordinate, *i.e.* 466.80046720. From the uncontracted calculations we obtain

```

> normal, mq= 0
M(e1) 1 1Sig+ 1 1Sig+ >= 466.80046120
M(e1) 2 1Sig+ 2 1Sig+ >= -533.19952827
M(e1) 3 1Sig+ 3 1Sig+ >= -533.19952767

```

which is what we expect. The second way to tell the difference is the degeneracy of the levels: at a bond length of 1000 a_o , the uncontracted energies are

```

mrci
1Sig+ > 1 -128.9810140593
1Sig+ > 2 -128.9512392537
1Sig+ > 3 -128.9512036298

```

Roots 2 and 3 are not exactly degenerate: we attribute this to the interacting space approximation, but clearly the first root is non-degenerate, what is what we expect for $N+O^+$.

We can also view this from the point of view of angular momentum coupling: Coupling the electronic angular momentum of two S functions gives an angular momentum of 0, while coupling the electronic angular momentum of two P functions gives angular momenta 0, 1 and 2. These coupled angular momenta are reflected in the following matrix elements, again from the uncontracted calculation:

```

< 1 1Sig+|Ax^2+Ay^2| 1 1Sig+>= 3.4794647E-03
< 2 1Sig+|Ax^2+Ay^2| 2 1Sig+>= 2.0008797E+00
< 2 1Sig+|Ax^2+Ay^2| 3 1Sig+>= -2.8281985E+00
< 3 1Sig+|Ax^2+Ay^2| 3 1Sig+>= 4.0005173E+00

```

Root one gives a result near 0, appropriate for $N+O^+$, while if we diagonalize the two by two matrix of the results for roots two and three, the eigenvalues

are 9.7E-04 and 6.000423E+00, corresponding to angular momenta 0 and 2 ($2(2+1)=6$), respectively from the $N^+ O$ asymptote. The function of $N^+ O$ with angular momentum 1 falls in the Σ^- symmetry block.

When we naively run a single root, the second calculation in the script we obtain the i^2cMRCI energy

```
1Sig+ > 1 -128.9487147564 -128.9642456058 -128.9642351358 -128.9532442217 -128.7068882570
-0.2418264994 0.9693567920 0.9396908193 -128.7068445232
```

and dipole

```
M(e1) 1 1Sig+ 1 1Sig+ >= -533.19952789
```

and angular momentum

```
< 1 1Sig+|Ax^2+Ay^2| 1 1Sig+>= 4.0004418E+00
```

The dipole and angular momentum values are indicative of $N^+ O$.

If we use the MRCI-S vectors to generate the contraction, the fourth calculation in the script, we obtain the i^2cMRCI energy

```
1Sig+ > 1 -128.9701164808 -128.9834536201 -128.9834536201 -128.9720958664 -128.8705070686
-0.0996094122 0.9391040232 0.8819163663 -128.6357256882
```

and dipole moment

```
M(e1) 1 1Sig+ 1 1Sig+ >= 466.80046165
```

Matrix elements of Ax^2+Ay^2 are not printed for this calculation, because the absolute value is less than $1d-10$. These are all indicative of $N O^+$, which is what we desire.

On my Mac M2 laptop, the MRCI property calculation fails because of a compiler issue.

11.8 C_2 molecule

The script that runs this calculation is `runc2.sample`.

In this test we show how one can use reference vectors of other symmetries to generate the contraction coefficients in such a way that all states of a given spin correlating to a give pair of atomic asymptotes are degenerate. It should be noted that the states on the *xref* line are not in addition to the state on the state line, but rather the complete list.

```
. ~/.premprec # set up mpec environment
echo "C2 sample"
shellguess=h0
/bin/rm summary.c2 # start with clean slate
for shellr in 100. # loop over geometries
do
echo "for bond length" $shellr
output=c2out`date -Idate`$shellr
/bin/rm $output
cat > input <<@
&BG
ends(1)=a to b
r $shellr
C is a,b
cc-pvtz
&CAS
guess $shellguess # generate starting orbitals from one-electron Hamiltonian
states
3P 3P
@
/bin/rm forbs # delete any orbitals lying about
runmprec < input >> $output
```

```

/bin/rm prop.in
gg=g
gu=g
ug=u
for shellspin in 1 3 5
do
for shellstate in Sig+$gu,2 Sig-$ug Pig Piu Del$gu
do
cat > input <<@
&MRCI
state $shellspin$shellstate
dynw 0.1 -75.4 # dynamic weights to soft pedal repulsive states when they split
xref Sig+$gu 2 Sig-$ug Pig Piu Del$gu
@
/bin/rm summary
runmpc < input >> $output
cat summary >> summary.c2
if [ $? -ne 0 ] ; then exit ; fi # bail if calculation failed
mv wavef wave$shellspin$shellstate$shellr
echo wave$shellspin$shellstate$shellr >> prop.in # build up prop input
done # end loop over symmetries
if [ $gu = $gg ]
then
gu=u
ug=g
else
gu=g
ug=u
fi
done
cat > input <<@
&PROP
@
cat prop.in >> input
runmpc < input >> $output
grep ' >' $output > $output.precis
shellguess=readin
done # end of loop over geometries.

```

The summary file from this run

Bond Length	avg MCHF energy	no. double excitations	State	CI Energies	
1.0000000E+02	-7.5408045E+01	59649	1Sig+g	-75.554402819	-75.5544028162
1.0000000E+02	-7.5408045E+01	59103	1Sig-u	-75.554402817	
1.0000000E+02	-7.5408045E+01	59199	1Pig	-75.5544028197	
1.0000000E+02	-7.5408045E+01	59199	1Piu	-75.5544028185	
1.0000000E+02	-7.5408045E+01	59649	1Delg	-75.5544028168	
1.0000000E+02	-7.5408045E+01	95316	3Sig+u	-75.5549088830	-75.5549088806
1.0000000E+02	-7.5408045E+01	94427	3Sig-g	-75.554908881	
1.0000000E+02	-7.5408045E+01	94691	3Pig	-75.554908883	
1.0000000E+02	-7.5408045E+01	94691	3Piu	-75.554908884	
1.0000000E+02	-7.5408045E+01	95316	3Delu	-75.5549088812	
1.0000000E+02	-7.5408045E+01	72165	5Sig+g	-75.554985294	-75.554985292
1.0000000E+02	-7.5408045E+01	71376	5Sig-u	-75.554985292	
1.0000000E+02	-7.5408045E+01	71617	5Pig	-75.554985295	
1.0000000E+02	-7.5408045E+01	71617	5Piu	-75.554985294	
1.0000000E+02	-7.5408045E+01	72165	5Delg	-75.554985292	

shows that the singlets all have the same energy, the triplets all have the same energy and the quintets all have the same energy, but the triplet energy is 0.5 mE_h lower than the singlet energy and the quintet energy is $76 \mu\text{E}_h$ lower than the triplet energy. These differences are due to the internal contraction: in uncontracted calculations, all spins would have the same energies.

It should be noted that when I run this example on different hardware, I obtain different property matrix elements. This is because the vectors of the degenerate eigenvalues of the same spin/symmetry block can mix arbitrarily. If one were to diagonalize the $A_x^2 + A_y^2$ matrix, this mixing would be uniquely (apart from a sign) resolved.

11.9 CH₂ molecule

In this test we exercise derivative routines. In the first CH₂ example, we compute the z component of the dipole moment both by finite difference with a finite

field, and by expectation value. We do this at the CAS level, because the results should be the same.

The script that runs this calculation is `runch2d.sample`.

```
. ~/.premprec                # set up mpec environment
for shellrel in off on        # loop over relativity
do
  output=3ch26`date -Idate`$shellrel.sample
  /bin/rm $output ff$output
  guess=h0                    # cold start for first calculation
  ifield=0                    # code for first field
  for shellfield in 0 0.005 -0.005 0.0025 -0.0025 # loop over fields
  do
    cat > input <<@
    &BG
    ffield z $shellfield      # the ffield command can be anywhere in the BG input
    rel $shellrel              # except between an atom and its basis designation
    ends(1)= a to bc, ends(2)= b to c
    rs 1.3,3.3
    thetad 90
    C is a
    cc-pvtz
    H is b,c
    cc-pvtz
    &CAS
    guess $guess
    States
    3B2
    &MRCI
    singx off                  # turn off singles and doubles
    doubx off
    @
    /bin/rm forbs              # delete left over orbitals
    runmpc < input >> $output
    cp forbs orbs              # copy orbitals for initial guess for next field
    if [ $? -ne 0 ] ; then exit ; fi # bail if orbitals didn't converge
    cat summary >> ff$output
    if [ $? -ne 0 ] ; then exit ; fi # bail if MRCI did not converge
    if [ $ifield -eq 0 ]        # for first field
    then
      cat > input <<@
      &PROP                    # compute dipole as an expectation value
      sotm off
      wavef
      @
      runmpc < input >> $output
      fi
      guess=readin             # reset guess
      ifield=1                 # reset field counter
      done                     # end field loop
      grep '>' $output > $output.precis
      done                     # end rel loop
```

For the non-relativistic case, the dipole moment computed as an expectation value is

```
< 1 3B2 |mu0 | 1 3B2 >= 6.3996902E-01
```

and the summary file `ff$output` is

1.3000000000000000	3.2999999999999998	1.5707963267948966	3.0000000000000000
0.0000000000000000	-38.694321863765325	48 3B2	-38.694321863779166
1.3000000000000000	3.2999999999999998	1.5707963267948966	3.0000000000000000
5.000000000000001E-003	-38.691255183959854	48 3B2	-38.691255183986712
1.3000000000000000	3.2999999999999998	1.5707963267948966	3.0000000000000000
-5.000000000000001E-003	-38.697654860309711	48 3B2	-38.697654860311758
1.3000000000000000	3.2999999999999998	1.5707963267948966	3.0000000000000000
2.500000000000001E-003	-38.692755226838671	48 3B2	-38.692755226840077
1.3000000000000000	3.2999999999999998	1.5707963267948966	3.0000000000000000
-2.500000000000001E-003	-38.695955076260802	48 3B2	-38.695955076305381

where the first three columns specify the geometry, column four is the field type (3 for *z* component), column five is the field strength, column six is the CAS energy, column seven is the number of CSFs, column eight is the state name, and column 9 is the energy from the MRCI routine. If the `ffield` keyword is not in the input, columns 4 and 5 are omitted. The field type number is echoed in the MPEC output where the field is parsed. If I do central difference derivatives,

I get 0.639967633 with fields $\pm 5 \times 10^{-3}$, 0.639969893 with fields $\pm 2.5 \times 10^{-3}$, and extrapolating to zero field, 0.639970647.

Similarly for the relativistic case, we get

```
< 1 3B2      |mu0 | 1 3B2      >= 6.3989329E-01

1.3000000000000000      3.2999999999999998      1.5707963267948966      3.0000000000000000
0.0000000000000000      -38.710343851801198      48 3B2      -38.710343851812389
1.3000000000000000      3.2999999999999998      1.5707963267948966      3.0000000000000000
5.0000000000000001E-003 -38.707277589512465      48 3B2      -38.707277589539800
1.3000000000000000      3.2999999999999998      1.5707963267948966      3.0000000000000000
-5.0000000000000001E-003 -38.713676505786594      48 3B2      -38.713676505788591
1.3000000000000000      3.2999999999999998      1.5707963267948966      3.0000000000000000
2.5000000000000001E-003 -38.708777414067384      48 3B2      -38.708777414068805
1.3000000000000000      3.2999999999999998      1.5707963267948966      3.0000000000000000
-2.5000000000000001E-003 -38.711976883828953      48 3B2      -38.711976883874293
```

and for central difference derivatives, I get 0.639891625 with fields $\pm 5 \times 10^{-3}$, 0.639893961 with fields $\pm 2.5 \times 10^{-3}$, and extrapolating to zero field, 0.639894740.

A good measure of the errors in the finite difference calculations is the difference between the extrapolated value and the value from the smallest stepsize: this is about 1×10^{-6} . In contrast, the difference between the extrapolated value and the value computed as an expectation value is about ten times larger. What does this mean? It means that to obtain accurate expectation values, it is necessary to more accurately converge the wavefunction, for the difference between the dipole from the derivative and the expectation value involves terms proportional to derivatives of the wavefunction with respect to wavefunction parameters, and these terms are not zero if we only converge the energy. If we decrease the gradient convergence parameter *gconv* we see that the expectation value actually converges to the extrapolated finite field value.

In the next CH₂ example, we analytically compute gradients of the MCHF energy and possibly the MCHF wavefunction. We test the three *grad* options: *e*, just compute the gradients of the MCHF energy with respect to geometric variables; no argument, just compute the totally symmetric gradients of the MCHF with respect to dipole components and geometric variables; and *bodc*, also compute the Born-Oppenheimer Diagonal Correction. These calculations are done with and without special relativity, and with and without symmetry. Furthermore, the without symmetry calculations use the converged orbitals from the calculation with symmetry as initial guess orbitals. The script that runs this calculation is *runch2.sample*.

```
. ~/.premprec      # set up mpec environment
for shellrel in off on      # loop over relativity
do
shellguess=h0      # for first pass, guess orbitals from h0
for shellsym in \# sym=off      # loop over symmetry on and off
do
echo "for sym " $shellsym
output=3ch2'date -Idate'$shellrel$shellsym
/bin/rm $output
for shellgrad in e \# bodc      # loop over grad types
do
echo "for grad type" $shellgrad
cat > input <<@
&BG
$shellsym
rel $shellrel
ends(1)= a to bc, ends(2)= b to c # C+H2 Jacobi coordinates
rs 1.3,3.3
thetad 90
C is a
cc-pvdz
H is b,c
```

```

cc-pvdz
&CAS
guess $shellguess
gconv 1d-7                                # note tighter gradient convergence criterion
states
3B1                                         # sym will default to 1 for no symmetry case
grad $shellgrad
@
/bin/rm forbs                             # delete left over orbitals
runmpc < input >> $output
mv forbs orbs                             # rename orbs for next guess:
if [ $? -ne 0 ]                           # bail if no final orbitals
then
exit
fi
shellguess=readin
done                                       # end grad type loop
grep ' >' $output > $output.precis      # short summary
done                                     # end symmetry loop
done                                     # norel/rel loop

```

The non-relativistic gradients computed with *grad e* are

```

>for operator d/dx 2- 3 with symmetry 1
de from genegy> 3.7054396987086236E-002
>for operator d/dz 1 with symmetry 1
de from genegy> -4.2402934582448815E-002
>for operator d/dz 2+ 3 with symmetry 1
de from genegy> 4.2402934582349339E-002

```

while those computed including the wavefunction derivatives are

```

>for operator d/dx 2- 3 with symmetry 1
1 3B1 > 3.705439699161E-02
>for operator d/dz 1 with symmetry 1
1 3B1 > -4.240293465256E-02
>for operator d/dz 2+ 3 with symmetry 1
1 3B1 > 4.240293465246E-02

```

that is an agreement of a few parts per 10^9 . If we hadn't reduced the convergence parameter *gconv* to 10^{-7} , the agreement from the results using *grad e* would not have been as good: see the discussion above about dipole gradients. The BODC computed using symmetry is

```

>for nucleus 1 36.637911497147563
>for nucleus 2 0.81003769080960741
>for nucleus 3 0.81003769080960741

```

while that computed without symmetry is

```

>for nucleus 1 36.637820744940697
>for nucleus 2 0.81003532183022608
>for nucleus 3 0.81003532183022742

```

These agree to a few parts per million.

When we turn on special-relativity we obtain

```

>for operator d/dx 2- 3 with symmetry 1
de from genegy> 3.6977508127739256E-002
>for operator d/dz 1 with symmetry 1
de from genegy> -4.2496880517076230E-002
>for operator d/dz 2+ 3 with symmetry 1
de from genegy> 4.2496880516977642E-002

```

and

```

>for nucleus 1 36.762007040943566
>for nucleus 2 0.81023061763298387
>for nucleus 3 0.81023061763298387

```

In the next CH_2 example, we compute the second order Born-Oppenheimer corrections using a Hartree-Fock wavefunction. We will compute these both with symmetry and without symmetry.

The script that runs this calculation is *run1ch2.sample*.

```

. ~/.premprec # set up mpec environment
basis=cc-pvdz
for shellrel in off on # loop over relativity
do
shellguess=h0 # initial guess eigenvectors of h0
for shellsym in \#sym=off sym=off # loop over symmetry on or off. For symmetry on, comment out
do # sym=off line with #, but need \ to prevent shell from reading
output=1ch2`date -Idate`$shellrel$shellsym ## as the start of a comment.
/bin/rm $output
cat > input <<@
&BG
rel $shellrel
C
0. 0. 0.
$basis
H
1.644403 0. 1.32213
-1.644403 0. 1.32213
$basis
$shellsym
&SOHF
guess $shellguess
grad noad
@
/bin/rm forbs # delete left over orbitals
runmprec < input >> $output
mv forbs orbs # rename orbs for guess for next calculation
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
shellguess=readin # reset shellguess
grep '>' $output > $output.precis # short summary
done # end loop over symmetry
done # end loop over relativity

```

The first order correction adds the function $\sum_i BODC_i/m_i$ to the potential energy surface, where the sum is over nuclei, m_i is the mass for nuclei i , and $BODC$ is the quantity MPEC computes.

Without special relativity, from the no symmetry calculation, we obtain for the diagonal correction

```

>bodc for nucleus no.      1   36.311615595729258
>bodc for nucleus no.      2   0.54057362003633391
>bodc for nucleus no.      3   0.54057362003632481

```

while for the calculation using C_{2v} symmetry, we obtain

```

>bodc for nucleus no.      1   36.311615601224041
>bodc for nucleus no.      2   1.0811472340717698
>bodc for nucleus no.      3   0.00000000000000000

```

While in the C_{2v} calculation, I could average the bodc over nuclei 2 and 3 to reproduce the result from no symmetry, I do not for the reason discussed below.

The second order correction adds the operator

$$\sum_{\beta i \beta' i'} \frac{\partial^\dagger}{\partial X_{\beta i}} \frac{C(2)_{\beta i \beta' i'}}{m_i m_{i'}} \frac{\partial}{\partial X_{\beta' i'}} + \sum_{\beta i i'} \frac{\partial^\dagger}{\partial X_{\beta i}} \frac{C(1)_{i' i \beta}}{m_i m_{i'}} + \sum_{\beta i i'} \frac{C(1)_{i' i \beta}}{m_i m_{i'}} \frac{\partial}{\partial X_{\beta i}} + \sum_{i i'} \frac{C(0)_{i' i}}{m_i m_{i'}} \quad (1)$$

to the nuclear Hamiltonian, where $\beta = x, y, z$ and $X_{\beta i}$ is the nuclear coordinate, and $\dagger$ means act to the left.

In the C_{2v} calculations, I compute derivatives with respect to the symmetry adapted coordinates, and to fit the functions $C(2)$, $C(1)$, and $C(0)$ as a function of geometry, I have transformed these quantities from the symmetry adapted coordinates to regular coordinates. These are denoted “desymmetrized $C(2)$ ” and “desymmetrized $C(1)$ ”. Then we can directly compare to the results from the no symmetry calculation. For the start of $C(2)$, we obtain from the no symmetry calculation

```

>C(2) matrix:
> x 1   x 1  -3.0650844E+00
> x 2   x 1   8.2330194E-01
> z 2   x 1  -5.3456991E-01
> x 3   x 1   8.2330194E-01
> z 3   x 1   5.3456991E-01
> y 1   y 1   1.0300314E+01
> y 2   y 1  -5.2787189E+00
> y 3   y 1  -5.2787189E+00
> z 1   z 1  -1.3529810E+00
.
.
.

```

and from the C_{2v} calculations,

```

>desymmetrized C(2)
> x 1   x 1  -3.0650840E+00
> x 2   x 1   8.2330176E-01
> z 2   x 1  -5.3456980E-01
> x 3   x 1   8.2330176E-01
> z 3   x 1   5.3456980E-01
> y 1   y 1   1.0300315E+01
> y 2   y 1  -5.2787192E+00
> y 3   y 1  -5.2787192E+00
> z 1   z 1  -1.3529810E+00
.
.
.

```

These agree quite well.

For C(1), from the no symmetry calculation, we obtain

```

>C(1)
> 1   z 1  -5.4867642E-01
> 1   x 2  -2.1399867E-01
> 1   z 2  -2.0970539E-01
> 1   x 3   2.1399867E-01
> 1   z 3  -2.0970539E-01
> 2   x 1  -2.5191077E-03
> 2   z 1   1.5064529E-01
> 2   x 2   2.0554900E-02
> 2   z 2  -8.4902706E-02
> 2   x 3   6.6991305E-02
> 2   z 3   6.1497045E-02
> 3   x 1   2.5191077E-03
> 3   z 1   1.5064529E-01
> 3   x 2  -6.6991305E-02
> 3   z 2   6.1497045E-02
> 3   x 3  -2.0554900E-02
> 3   z 3  -8.4902706E-02

```

and from the calculation using C_{2v} symmetry,

```

>desymmetrized C(1)
> 1   z 1  -5.4867642E-01
> 1   x 2  -2.1399866E-01
> 1   z 2  -2.0970539E-01
> 1   x 3   2.1399866E-01
> 1   z 3  -2.0970539E-01
> 2   z 1   1.5064529E-01
> 2   x 2  -2.3218202E-02
> 2   z 2  -1.1702830E-02
> 2   x 3   2.3218202E-02
> 2   z 3  -1.1702830E-02
> 3   z 1   1.5064529E-01
> 3   x 2  -2.3218202E-02
> 3   z 2  -1.1702830E-02
> 3   x 3   2.3218202E-02
> 3   z 3  -1.1702830E-02

```

While it appears that there is some disagreement between these two results, *e.g.* the entries

```

> 2   x 1  -2.5191077E-03
> 3   x 1   2.5191077E-03

```

are missing from the C_{2v} results, for a system where the mass for nucleus 2 is the same as the mass as nucleus 3, these entries will cancel. Similarly, the average of the entries

```
> 2 x 2 2.0554900E-02
> 3 x 2 -6.6991305E-02
```

agrees with what the C_{2v} calculation produces. Similarly, for $C(0)$, the no symmetry calculation yields

```
>C(0)
> 1 1 -7.4361356E+00
> 2 1 1.4305811E-01
> 3 1 1.4305811E-01
> 1 2 1.4305811E-01
> 2 2 -3.3662027E-01
> 3 2 -3.5849040E-02
> 1 3 1.4305811E-01
> 2 3 -3.5849040E-02
> 3 3 -3.3662027E-01
```

while the results from the C_{2v} calculation are

```
>C(0)
> 1 1 -7.4361356E+00
> 2 1 1.4305810E-01
> 3 1 1.4305810E-01
> 1 2 1.4305810E-01
> 2 2 -1.8623465E-01
> 3 2 -1.8623465E-01
> 1 3 1.4305810E-01
> 2 3 -1.8623465E-01
> 3 3 -1.8623465E-01
```

If we average

```
> 2 2 -3.3662027E-01
> 3 2 -3.5849040E-02
> 2 3 -3.5849040E-02
> 3 3 -3.3662027E-01
```

we reproduce the results from the C_{2v} calculations.

Thus we see if the masses of the two equivalent nuclei are the same, the C_{2v} calculations reproduce the second order correction to the nuclear Hamiltonian given by the no symmetry calculation. However, if the masses are different, the results of the C_{2v} calculation will NOT reproduce the second order correction to the nuclear Hamiltonian given by the no symmetry calculations. This occurs because I do not consider second derivative operators that are not totally symmetric because the cross derivatives cancel out if the masses of the two equivalent nuclei are the same. This is error on my part.

This is only an issue if including symmetry results in pairs of nuclei that are related by a symmetry operator.

When we include special relativity, we obtain very similar results, so we only quote C_{2v} results:

```
>desymmetrized C(2)
> x 1 x 1 -3.0639524E+00
> x 2 x 1 8.2300874E-01
> x 2 x 1 -5.3464379E-01
> x 3 x 1 8.2300874E-01
> x 3 x 1 5.3464379E-01
> y 1 y 1 1.0291017E+01
> y 2 y 1 -5.2747023E+00
> y 3 y 1 -5.2747023E+00
> z 1 z 1 -1.3530627E+00
.
.
.
>desymmetrized C(1)
> 1 z 1 -5.4987320E-01
> 1 x 2 -2.1390627E-01
> 1 z 2 -2.0993436E-01
> 1 x 3 2.1390627E-01
> 1 z 3 -2.0993436E-01
> 2 z 1 1.5039501E-01
```

```

> 2 x 2 -1.8163138E-02
> 2 z 2 -5.1513204E-03
> 2 x 3 1.8163138E-02
> 2 z 3 -5.1513204E-03
> 3 z 1 1.5039501E-01
> 3 x 2 -1.8163138E-02
> 3 z 2 -5.1513204E-03
> 3 x 3 1.8163138E-02
> 3 z 3 -5.1513204E-03

```

and

```

>c(0)
> 1 1 -7.4799665E+00
> 2 1 1.4643102E-01
> 3 1 1.4643102E-01
> 1 2 1.4643102E-01
> 2 2 -1.7948066E-01
> 3 2 -1.7948066E-01
> 1 3 1.4643102E-01
> 2 3 -1.7948066E-01
> 3 3 -1.7948066E-01

```

In the final CH₂ example, we compute the BODC correction for multiple roots and since there are multiple roots, there are also non-zero first derivative coupling terms, and we compute them as well. The C_{2v} calculation uses the script `runch2mrc2v.sample`:

```

. ~/.premppec # set up mpec environment
output=3ch2mrootc2v'date -Idate'
/bin/rm $output
cat > input <<@
&BG
ends(1)= a to bc, ends(2)= b to c # C+H2 Jacobi coordinates
r 1.3,3.3
thetad 90
C is a
cc-pvdz
H is b,c
cc-pvdz
&CAS
guess
gconv 1d-7 # note tighter gradient convergence criterion
states
3B1 2 1. 1.
3A2 1 1.
3B2 2 1. 1.
3A1 1 1.
grad bodc
@
/bin/rm forbs # delete left over orbitals
runmppec < input >> $output
mv forbs orbs # rename orbs for next guess:
if [ $? -ne 0 ] ; then exit ; fi # bail if no final orbitals
grep '>' $output > $output.precis # short summary

```

while the C₁ calculation uses the script `runch2mrc1.sample`:

```

. ~/.premppec # set up mpec environment
output=3ch2mrootc1'date -Idate'
/bin/rm $output
cat > input <<@
&BG
sym=off
ends(1)= a to bc, ends(2)= b to c # C+H2 Jacobi coordinates
r 1.3,3.3
thetad 90
C is a
cc-pvdz
H is b,c
cc-pvdz
&CAS
guess
gconv 1d-7 # note tighter gradient convergence criterion
states
3S1 6
grad bodc
@
/bin/rm forbs # delete left over orbitals
runmppec < input >> $output
mv forbs orbs # rename orbs for next guess:
if [ $? -ne 0 ] ; then exit ; fi # bail if no final orbitals
grep '>' $output > $output.precis # short summary

```

On my computer, the relation between the no symmetry and C_{2v} wavefunctions is $1\ 3S1 = -1\ 3B1$, $2\ 3S1 = -1\ 3A2$, $3\ 3S1 = -1\ 3B2$, $4\ 3S1 = 1\ 3A1$, $5\ 3S1 = -2\ 3B2$, and $6\ 3S1 = 2\ 3B1$, and for the first derivatives, the results are the same between the no symmetry and C_{2v} calculations. The first derivative matrix elements are skew symmetric, and totally symmetric operators only couple states of the same symmetry, *e.g.*

```
< 2 3B1 |d/dz 1 1 3B1 > = -5.6819604E-02 = 1.0270136E-01 and -1.5952096E-01
< 2 3B2 |d/dz 1 1 3B2 > = -7.6583963E-02 = -7.8235848E-02 and 1.6518846E-03
```

and operators of other symmetries couple states of different symmetries, *e.g.*

```
< 1 3B1 |d/dx 2+ 3 1 3A2 > = -1.8497758E-01 = -1.8021304E-01 and -4.7645398E-03
< 2 3B1 |d/dx 2+ 3 1 3A2 > = -2.0510763E-01 = -2.3807727E-01 and 3.2969636E-02
< 1 3B2 |d/dx 2+ 3 1 3A1 > = 2.2221023E+00 = 2.2276605E+00 and -5.5582021E-03
< 2 3B2 |d/dx 2+ 3 1 3A1 > = -2.4970689E+00 = -2.5047658E+00 and 7.6969259E-03
```

In these results, after the second equal sign, we decompose the results into the implicit contribution, change in eigenvectors and change in MO vectors in the orthogonal basis, and the explicit contribution, change due to the basis functions being attached to the nuclei.

For the BODC correction, for the reasons discussed above, the C_{2v} calculations do not generate contributions between states of different symmetries, *e.g.*

```
>states 2 6 of 3S1
>for nucleus 1 -1.0525006490177835E-010
>for nucleus 2 -1.9973164132200472
>for nucleus 3 1.9973164131953047
```

which cancels out if the masses of nucleus 2 and nucleus 3 are the same.

11.10 O₂

The script that runs this calculation is runo2.sample.

In this example we exercise the i²cMRCI gradient routines.

```
. ~/.prempc # set up mpec environment
state=3Sig-g
for shellnuc in 160 180 # loop over iso is loop over d2h/c2v
do
for shellrel in off on # loop over relativity
do
output=o$shellnuc$shellrel'date -Idate'
/bin/rm $output
cat > input <<@
&BG
rel $shellrel
ends(1)= a to b
rs 2.
0 is a
cc-pvdz
$shellnuc is b
cc-pvdz
&CAS
maxitmac 20 # h0 guess is not very good, so allow extra iterations
guess h0
states
$state
grad # turn on gradient calculation
&MRCI
wconv id-8 # invoke wavefunction convergence option
@
/bin/rm forbs borbs summary # delete left overs
runmpc < input >> $output
mv forbs orbs # rename orbs for next guess:
if [ $? -ne 0 ] ; then exit ; fi # bail if no final orbitals
grep '>' $output > $output.precis # short summary
done # end loop over relativity
state=3Sig-
done
```

This yields

	non-relativistic	relativistic
016		
Q0 field	-6.96015197519082E-01	-6.92026052703192E-01
ReQ2 field	2.68470408944205E-11	2.64836592948477E-11
d/d z 1- 2	8.55253314212488E-01	8.54658951592976E-01
018		
muz field	4.73915619473843E-10	2.40010985320596E-10
Q0 field	-6.96015746172317E-01	-6.92026523803848E-01
ReQ2 field	1.64579376909877E-10	1.64784391928320E-10
d/dz 1	4.27626285575544E-01	4.27329178455444E-01
d/dz 2	-4.27626285574768E-01	-4.27329178455517E-01

In D_{2h} , MPEC computes the field gradients for the quadrapole moments Q0 and ReQ2, and the geometric gradient $\frac{\partial}{\partial z_1} - \frac{\partial}{\partial z_2}$, where z_i is the cartesian coordinate of atom i . Although ReQ2 transforms as A_g in D_{2h} , it does not transform as Σ_g^+ in $D_{\infty h}$, so the computed value is essentially zero. In C_{2v} , MPEC computes the field gradient of muz as well, and separately computes the geometric gradients $\frac{\partial}{\partial z_1}$ and $\frac{\partial}{\partial z_2}$. Once again, although muz transforms as A_1 in C_{2v} , it does not transform as Σ^+ in $C_{\infty v}$, so the computed value is essentially zero.

It is valuable to relate the cartesian derivatives to derivatives of the bond length, r . In the present example, we need to transform from $\frac{\partial}{\partial z_1}$ and $\frac{\partial}{\partial z_2}$ to $\frac{\partial}{\partial r}$ and $\frac{\partial}{\partial R}$ where $r = z_2 - z_1$ and $R = (m_1 z_2 + m_2 z_1)/m_{12}$, $m_{12} = m_1 + m_2$. Solving for the cartesian, we have $z_1 = R - m_1 r/m_{12}$ and $z_2 = R + m_2 r/m_{12}$. Thus $\frac{\partial}{\partial r} = -\frac{m_1}{m_{12}} \frac{\partial}{\partial z_1} + \frac{m_2}{m_{12}} \frac{\partial}{\partial z_2}$ and $\frac{\partial}{\partial R} = \frac{\partial}{\partial z_1} + \frac{\partial}{\partial z_2}$. Since $\frac{\partial E}{\partial R} = 0$, we have $\frac{\partial E}{\partial z_1} = -\frac{\partial E}{\partial z_2}$, so $\frac{\partial E}{\partial r} = \frac{\partial E}{\partial z_2} = -\frac{\partial E}{\partial z_1}$. Thus $\frac{\partial E}{\partial r} = -4.27626285574768E - 01$ in the non-relativistic C_{2v} case. In the D_{2h} case, we have $\frac{\partial E}{\partial r} = 0.5[\frac{\partial E}{\partial z_2} - \frac{\partial E}{\partial z_1}] = -0.5[\frac{\partial E}{\partial z_1} - \frac{\partial E}{\partial z_2}] = -0.5 \times 8.55253314212488E - 01 = -4.2762665710624E - 01$.

It should be noted that $r = z_2 - z_1$ applies to the situation where $z_2 > 0$ and $z_1 < 0$, which is the case with the input for this example. If, however, we had used ends(1)= b to a, then $z_2 < 0$ and $z_1 > 0$, so that $r = z_1 - z_2$ and the signs are reversed.

11.11 CH₄

The script that runs this calculation is `runch4.sample`.

In this example we test the geometry optimization routines. It should be noted that the vibrational coordinates used are somewhat convoluted. To maximize the equivalence of the Hydrogen nuclei, one would use the 5 nuclei Radau coordinates. However, since the z axis is aligned along the first vector, the calculation would only use C_s symmetry. By using the combination of Jacobi and Radau coordinates, we can make the z axis bisect a pair of Hydrogen nuclei, allowing us to use C_{2v} symmetry. In this example, we both search for the minimum and search for a stationary point.

```
. ~/.prempc # set up mpec environment
basis=cc-pvdz
for shelltype in min stationary
do
output=ch4'date -Idate '$shelltype.out'
/bin/rm $output
cat > input <<@
&BG
ends(1)=cde to ab, ends(2)=a to b, ends(3)=c to cde, ends(4)=d to cde
rs 1.5 3.24 2. 2.
thetad 90,54,54
```

```

phid 90,270
C is e
$basis
H is a,b,c,d
$basis
&CAS
guess
gconv 1d-7          # as in other examples, we use a tighter
states              # converged when relying on gradients computed
1S1                 # without solving the response equations.
grad $shelltype
@
runmpec < input >> $output
grep '>' $output > $output.precis
done

```

After 11 calls to the MCHF program, the minimization yields

```

>coordinates at minimum:
>rs      1.3879260587894073      3.4323985806176633      2.0508285405273847      2.0508285405273843
>theta   1.5707963267948966      0.99135256236519864      0.99135256236519875
>phi     1.5707963267948966      -1.5707963267948968

```

with energy -40.28104089 and root-mean-square norm of gradient 1.5×10^{-5} . In contrast, the search for the stationary point stops after 100 calls to the MCHF program with root-mean-square norm of the gradient 2.1×10^{-3} and energy -40.2809889 . If one searches backward in the output file for “< best so far”, the results for the best estimate for the stationary point can be found.

Note that apart from the > in the above output, the coordinates can be directly pasted into the MPEC input.

That fact that the minimization converged rapidly is because we use the variable metric algorithm, and this approaches second order convergence when sufficiently close to the minimum. In contrast, the stationary routine uses Powell’s method to find the minimum of the root-mean-square gradient norm, and thus is much less efficient.

It should be noted that the symmetry used will not change during the search for the minimum or stationary point. Thus if the minimum or stationary point does not have the same symmetry as the initial guess, then MPEC will not find the desired geometry. However, if the initial guess has less symmetry than the desired minimum or stationary point, then MPEC will converge to the desired geometry, but all calculations will be carried out using the lower symmetry.

11.12 N₂

The script that runs this calculation in runn2.sample

In this example we test the concatenation of orbital files of fragement for an initial guess for a supermolecule calculation.

```

. ~/.premppec
echo "N2 molecule sample"
for shellrel in off ss          # loop over relativity off or on
do
  output=n2out`date -Idate`$shellrel
  cat <<@ > input              # for N atom on the right.
  &BG
  rel $shellrel
  N
  0. 0. 50.
  cc-pvdz
  &CAS
  doub 2 0
  act 1 1 1
  guess h0
  states
  4S4

```

```

@
/bin/rm forbs
runmpec < input > $output
mv forbs orbs4Nr
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
cat <<@ > input # for N atom on the left.
&BG
rel $shellrel
N
0. 0. -50.
cc-pvdz
&CAS
doub 2 0
act 1 1 1
guess h0
states
4S4
@
/bin/rm forbs
runmpec < input >> $output
mv forbs orbs4Nl
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
cat <<@ > input # concatenate orbital file
&CAT
orbs4Nr
orbs4Nl
@
/bin/rm forbs
runmpec < input >> $output
mv forbs orbsN2rl
if [ $? -ne 0 ] ; then exit ; fi # bail if no orbitals
cp orbsN2rl orbs # supermolecule calculation in C2v
cat <<@ > input
&BG
ends(1)=a to b
rel $shellrel
r 100.
14N is a
cc-pvdz
15N is b #using different isotopologues forces asymmetry
cc-pvdz
&CAS
doub 4
act 2 2 2 #guess not required, for default is to read in guess
states
1A1
3A1
5A1
7A1
@
/bin/rm forbs
runmpec < input >> $output # supermolecule calculation in D2h
cat <<@ > input
&BG
ends(1)=a to b
rel $shellrel
r 100.
N is a,b
cc-pvdz
&CAS
doub 2 0 0 0 2
act 1 1 1 0 1 1 1 0
states
1Sig+g
3Sig+u
5Sig+g
7Sig+u
@
/bin/rm forbs
runmpec < input >> $output
done # end loop over relativity

```

We compute N atom orbitals at $\pm 50 a_o$ and store them in the files orbs4Nl and orbs4Nr. Then we use CAT to form the file for the supermolecule orbitals orbsN2rl. In the CAT output, the number of doubly occupied and active orbitals in the supermolecule is given, as well as the sum of the fragment energies, -108.83956380281822 E_h for the relativistic case.

We then carry two supermolecule calculations, one using C_{2v} symmetry and the other using D_{2h} symmetry. In both cases, the energy from the initial guess orbitals is -108.839564 E_h and the MCHF iterations converge after a single

iteration.

11.13 CN

The script that runs this calculation is `runcn.sample`.

In this example we test the MRCI restart option.

```
. ~/.premprec # set up mpec environment
echo "CN sample"
output=CNtest`date -Idate`.out
/bin/rm $output
cat > input <<@
&BG # generate orbitals
ends(1) a to b
r 2.
C is a
cc-pvtz
N is b
cc-pvtz
&CAS
guess
states
2Pi
@
runmprec < input >> $output
for shellsing in on off # test all possibilities
do
for shelldoub in \# off uc # test all possibilities
do
shellrestart=off # for ipass=1, no restart, but abort on
shellabort=6 # iteration 6
for ipass in 1 2
do
cat > input <<@
&MRCI
restart $shellrestart
itabort $shellabort
singx $shellsing
doubx $shelldoub
@
runmprec < input >> $output
shellrestart=yes # next time through restart
shellabort=0 # don't abort next time
done # end loop over restart off/on
done # loop over doubles on,off, uc
done # loop over singles on,off
```

11.14 HCN

The script that runs this calculation is `runhcn.sample`.

In this example we use Hartree-Fock orbitals as an initial guess and get the molecular geometry from a file. The script is quite sophisticated, allowing for more than one batch job to be working on a list of files, handling the case where the HF calculation did not converge, and saving the results to a master file. Furthermore, if the MRCI calculations did not converge, the geometry file is left starting with the letter b, but if it did converge, the geometry is left starting with the letter d. The HF calculation did not converge in this example, but the best orbitals in *borbs* still provides an excellent initial guess for the CAS calculation.

```
. ~/.premprec # set up mpec environment
echo "HCN sample"
cat > gfileaa <<@
r 5 2.4 # one would have some other means of generating a list of geometry files
thetad 120 # for example a single file of geometries separated by the split command.
@
basis="cc-pvtz &aug" # short hand for basis
# we get current list of top 10 files
for file in `ls gfile* | head -10`
do # rename file. If another batch job did
echo "file is " $file
```

```

mv $file b$file
if [ $? -eq 0 ]                # rename first then we return an
then                          # error, i.e. $? ne 0
output=hcn.test'date -Idate'$.file
/bin/rm $output
cat > input <<@
&BG
ends(1) a to bc, ends(2) b to c # define coordinates, H+NC Jacobis
grab b$file                    # get geometrical data from file
C is b
$basis
N is c
$basis
H is a
$basis
&SOHF                          # we are doing HF to generate initial guess orbitals
@
/bin/rm forbs borbs           # get rid of left overs
runmpec < input > $output
mv forbs orbs                 # plan a: converged orbitals
if [ $? -ne 0 ] ; then mv borbs orbs ; fi # plan b: best orbitals so far
cat > input <<@
&BG
ends(1) a to bc, ends(2) b to c
grab b$file
C is b
$basis
N is c
$basis
H is a
$basis
&CAS
states                         # default active space, default is read in guess
1A'                            # so just need states input
&MRCI                          # and MRCI does the same state as the cas
@
/bin/rm summary               # get rid of left overs
runmpec < input >> $output
cat summary >> masterlist     # save to master list
if [ $? -eq 0 ]
then
mv b$file d$file              # if things went ok, rename file from b to d.
mv forbs forbs$file          # save orbitals for next example
fi                             # if failed, leave b$file hanging around
fi                             # end copy if block
done                           # end loop over geometry files

```

The summary file for this run is

```

5.0000000E+00  2.4000000E+00  2.0943951E+00 -9.2856715E+01  225117 1A' -93.052667804

```

```

In comparison, for this same calculation MOLPRO obtained
!MCSHF STATE 1.1 Energy -92.856715417378
!MRCI STATE 1.1 Energy -93.052667831650

```

We believe the $0.03 \mu E_h$ difference between the MPEC and MOLPRO results is due to differences in the way the integrals are computed.

12 Release Notes

12.1 version theta

Fixed a bug causing incorrect i^2c MRCI-SD energies for a 2 electron singlet.

Fixed a bug relating to using core valence basis sets in relativistic calculations.

Fixed an error in the setting up of bond-length-bond-angle coordinates for triatomic molecules. Added the *bisect* options for orientating a triatomic molecule.

Added the CAT routine and fixed a bug in the orbital guess routine.

Fixed a bug in geometry optimization routines.

In the i²cMRCI Davidson diagonalization, the trial vectors are now normalized before they are orthogonalized from existing vectors. This seems to fix issues with the trial vectors losing linear independence.

Fixed a bug with the DIAB procedure that arose when there was only one orbital in a given class.

12.2 version eta

The major addition to this version of MPEC is the ability to analytically evaluate gradients of the i²cMRCI energies.

In addition, there have been numerous bug fixes:

A memory leak in the MCHF code has been fixed.

The integral routine used to generate derivatives of the nuclear attraction integrals for the finite nucleus (relativistic) case was not always very accurate. This has been corrected.

There was an error in diagonalizing the uncontracted MRCI-SD Hamiltonian if one symmetry had zero virtual orbitals. This has been corrected.

There was an error in computing $+q(sx)$ for more than one root. This has been corrected.

A bug in the spin-orbit integrals was fixed. I have seen it only affect the results of D_{2h} linear molecule calculations.

For linear molecules, there was an error printing out Spin-orbit matrix elements when either the Gaunt or Breit interaction was included. In addition, matrix elements with $\Sigma - \Sigma' = \pm 2$ were not computed. Also, the required matrix elements for IIII were not all computed.

12.3 version zeta

This is the first publish release of MPEC, and while the code is ready for many applications, there remain some outstanding issues.

The analytic calculation of derivatives only works with a determinant basis in the MCHF calculations, and does not work if eigenvalues of the overlap matrix are neglected in the orthogonalization of the AO basis.

The transformation to a diabatic orbital basis may not work if eigenvalues of the overlap matrix are neglected in the orthogonalization of the AO basis.

The analytic calculation of second derivative matrix elements when there are pairs of symmetry equivalent nuclei do not include second derivative operators that are not totally symmetric. The analytic computation of wavefunction gradients with respect to nuclear displacements is not possible for Rydberg orbitals.

13 Installation

The installation instructions are given in the README file, which is reproduced here:

Thank you for installing MPEC!
The copyright notice and disclaimers are given in the top of the file start.f in the source directory.

The distribution package consists of the follow files:

```
README      - this file
data.tar    - basis set and other data for MPEC
lib.tar     - math library directory
mpeclogo.pdf - MPEC logo
mpecman.pdf - user manual for MPEC
mfileh2.x   - script to make MPEC
source.tar  - MPEC source code directory
test.tar    - MPEC test run directory
```

To install MPEC, edit mfileh2.x to set the compilers, etc to create MPEC.
Then run using the Borne shell, i.e. sh mfileh2.x.

I include as examples creating MPEC on my laptop computer which has open-mpi installed on it and creating MPEC on the NAS facility parallel computers. Hopefully your situation will be similar to one of these two examples.

A script named .prempc is created in the installers home directory. It must be executed in every MPEC job script, as shown as the sample inputs. The sample inputs must be run using the Borne shell, i.e. sh runcontract.sample.

Some of the MPEC routines are sufficiently complex that compilers I have access to do not compile them properly with optimization turned on. Those routines use \$optno as their optimization flag while then other routines use \$opt as their optimization flag. Make sure to set both variables!

One has multiple options in the creation of MPEC:

One can use basic MPI commands (DDI-1) or MPI-4 commands (DDI-4) by setting the shell variable ddi to 1 (for DDI-1) and something else otherwise. For fewer than ~25 processors, use MPI-4.

One can use a system math library or a library created from the files in lib.tar: set the shell variable lib to 0 for the former choice and non-zero otherwise.
These library files are available under the modified BSD license, which is given at the end of this document.

One can use the matrix multiply routine dgemm from the library or one can use a special version that stops in case of an error and writes out where it is called: on the NAS systems the library dgemm writes out an error message but continues when an error is encountered. Set the shell variable dgemm to 0 to get the library matrix multiply routine and non-zero to get the special version.

Default values of parameters are set in a series of include files. If you don't like values I use, feel free to change them. This must be done before compilation.
The include files are in the source directory, and are
cas.par
mrci.par

Most dimensions in the program are allocated by reserving space in one massive array, but there are some dimensions set by parameter statements. These are mainly used to make input easier, and do not involve large amounts of memory, so large values are not detrimental.

The size of the massive array is specified by the parameter maxbc in the file common.store.
The current value is 2**31-1, the maximum memory that can be addressed with a signed 4 byte integer. If this exceeds your hardware limitations, that should not be an issue on modern computers unless MPEC tries to use more memory than you hardware allows. In the latter case, the program will crash with some sort of system error.

The maximum number of atoms is specified by the parameter ida in the file common.basis

The maximum number of spin/symmetry blocks in the state averaged cas is specified by the parameter maxst1 in the file common.cas

The maximum number of roots per spin/symmetry block in the state averaged cas is specified by the parameter maxst2 in the file common.cas.

The maximum number of reference orbitals in the MRCI calculations is specified by the parameter ido in the file common.mrci.

The maximum number of extra symmetries to generate contraction coefficients in the i²cMRCI calculations is specified by the parameter maxxref in the file common.mrci.

The maximum number of distributed arrays is specified by the parameter id in common.mympi.template.

For DDI-1 calculations, the maximum number of processors is specified in by parameter mxcpoc in common.mympi.template

Modified BSD license:

Like all software, it is copyrighted. It is not trademarked, but we do ask the following:

If you modify the source for these routines we ask that you change the name of the routine and comment the changes made to it.
We will gladly answer any questions regarding the software. If a modification is done, however, it is the responsibility of the user to modify the routine to provide support.

LICENSE

Copyright (c) 1992-2013 The University of Tennessee and The University
of Tennessee Research Foundation. All rights
reserved.

Copyright (c) 2000-2013 The University of California Berkeley. All

rights reserved.
Copyright (c) 2006-2013 The University of Colorado Denver. All rights reserved.

\$COPYRIGHT\$

Additional copyrights may follow

\$HEADER\$

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer listed in this license in the documentation and/or other materials provided with the distribution.
- Neither the name of the copyright holders nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

The copyright holders provide no reassurances that the source code provided does not infringe any patent, copyright, or any other intellectual property rights of third parties. The copyright holders disclaim any liability to any recipient for claims brought against recipient by any third party for infringement of that parties intellectual property rights.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.